Numerical and simulation techniques in finance

Nicolas Langrené

Beijing Normal-Hong Kong Baptist University

2025 Fall Semester

6.1 Simulation of stochastic process

Brownian motion

A standard **Brownian motion** (a.k.a. **Wiener process**) is a stochastic process $W = (W_t)_{0 \leq t \leq T}$ such that

$\triangleright$ $W_0 = 0$
$\triangleright$ $t \mapsto W_t$ is almost surely (a.s.) **continuous**
$\triangleright$ W has **independent increments**: for any $0 \leq s_1 \leq t_1 \leq s_2 \leq t_2$, the increments $W_{t_2} - W_{s_2}$ and $W_{t_1} - W_{s_1}$ are independent
$\triangleright$ W has **Gaussian increments**: for any $0 \leq s \leq t$, the increment $W_t - W_s$ follows a Gaussian distribution with mean 0 and variance $t - s$

$$W_t - W_s \sim \mathcal{N}(0, t - s)$$

Simulate a Brownian motion

In order to simulate a stochastic process, we first need to perform a time discretization. Introduce the time grid $\mathcal{T} = \{t_0, t_1, \ldots, t_N\}$ such that $t_n = n \times \Delta_t$ where $\Delta_t := T/N$. Drawing a Monte Carlo path from a Brownian motion can be achieved as follows:

▷ Simulate N independent draws G_n, $n = 1, \ldots, N$ from a standard Gaussian variable

▷ Set $W_{t_0} = 0$ and for every $n = 0, 1, \ldots, N-1$, define

$$W_{t_{n+1}} = W_{t_n} + \sqrt{\Delta_t} G_n$$

Simulating multiple BM paths: nested loops

```
import time
import numpy as np
from scipy.stats import norm

T = 1.0 # time horizon (option maturity)
N = 100 # number of time steps
M = 50000 # number of Monte Carlo paths
dt = T/N # time step size
sqrtdt = np.sqrt(dt) # square root of time step
W = np.zeros([N+1,M]) # stores Brownian paths (row-major)
start_time = time.time()
for m in np.arange(M):
    # simulate paths one by one
    for n in np.arange(N):
        W[n+1,m] = W[n,m] + sqrtdt*norm.rvs()
print('Execution time in seconds: ' + str(time.time() - start_time))
## Execution time in seconds: 303.3052341938019
```

Simulating multiple BM paths: vectorization

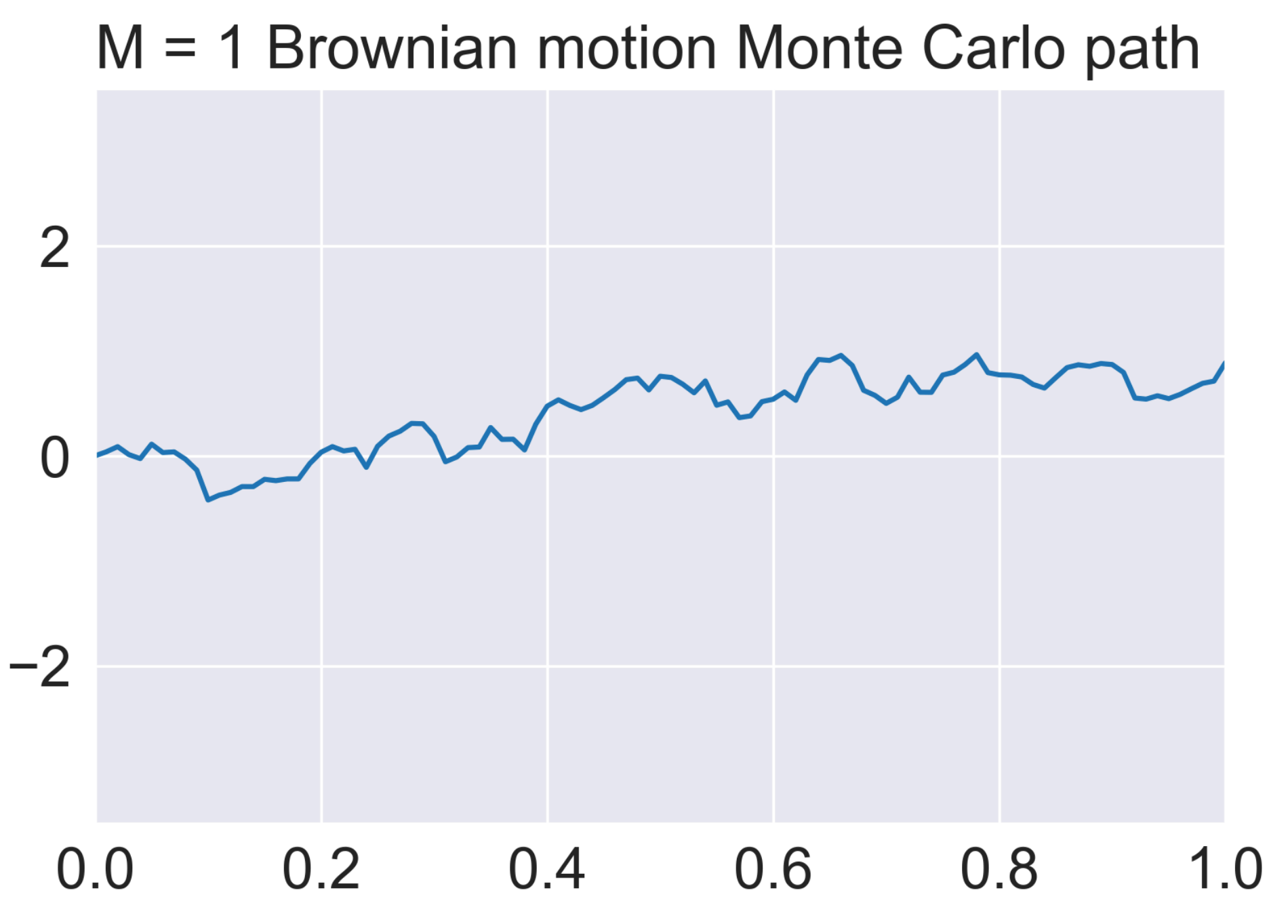
```
import time
import numpy as np
from scipy.stats import norm

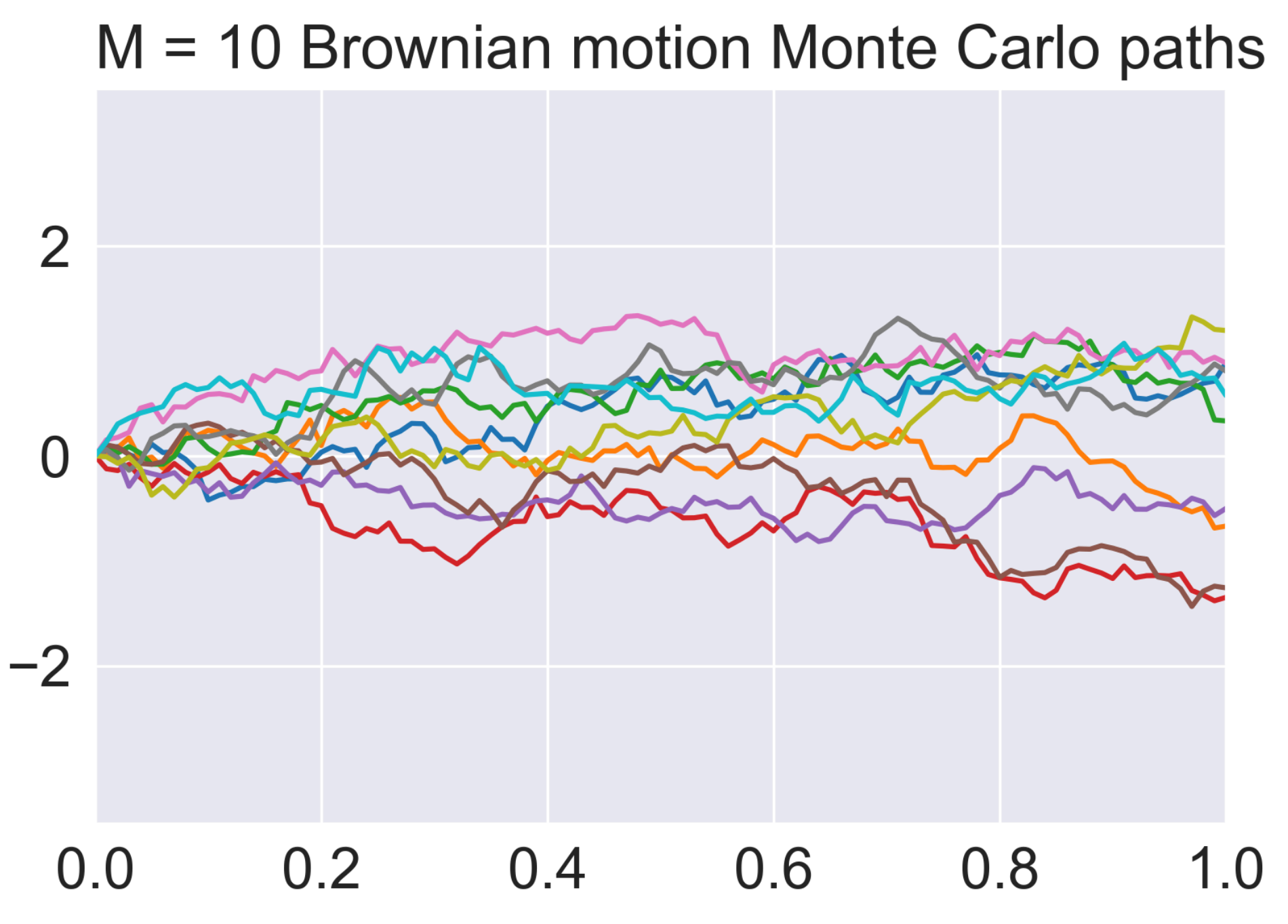
T = 1.0 # time horizon (option maturity)
N = 100 # number of time steps
M = 50000 # number of Monte Carlo paths
dt = T/N # time step size
sqrtdt = np.sqrt(dt) # square root of time step
W = np.zeros([N+1,M]) # stores Brownian paths (row-major)
start_time = time.time()
for n in np.arange(N):
    # simulate all paths simultaneously
    W[n+1,:] = W[n,:] + sqrtdt*norm.rvs(size=M)
print('Execution time in seconds: ' + str(time.time() - start_time))
## Execution time in seconds: 0.3546528816223144   # 900 times faster
```

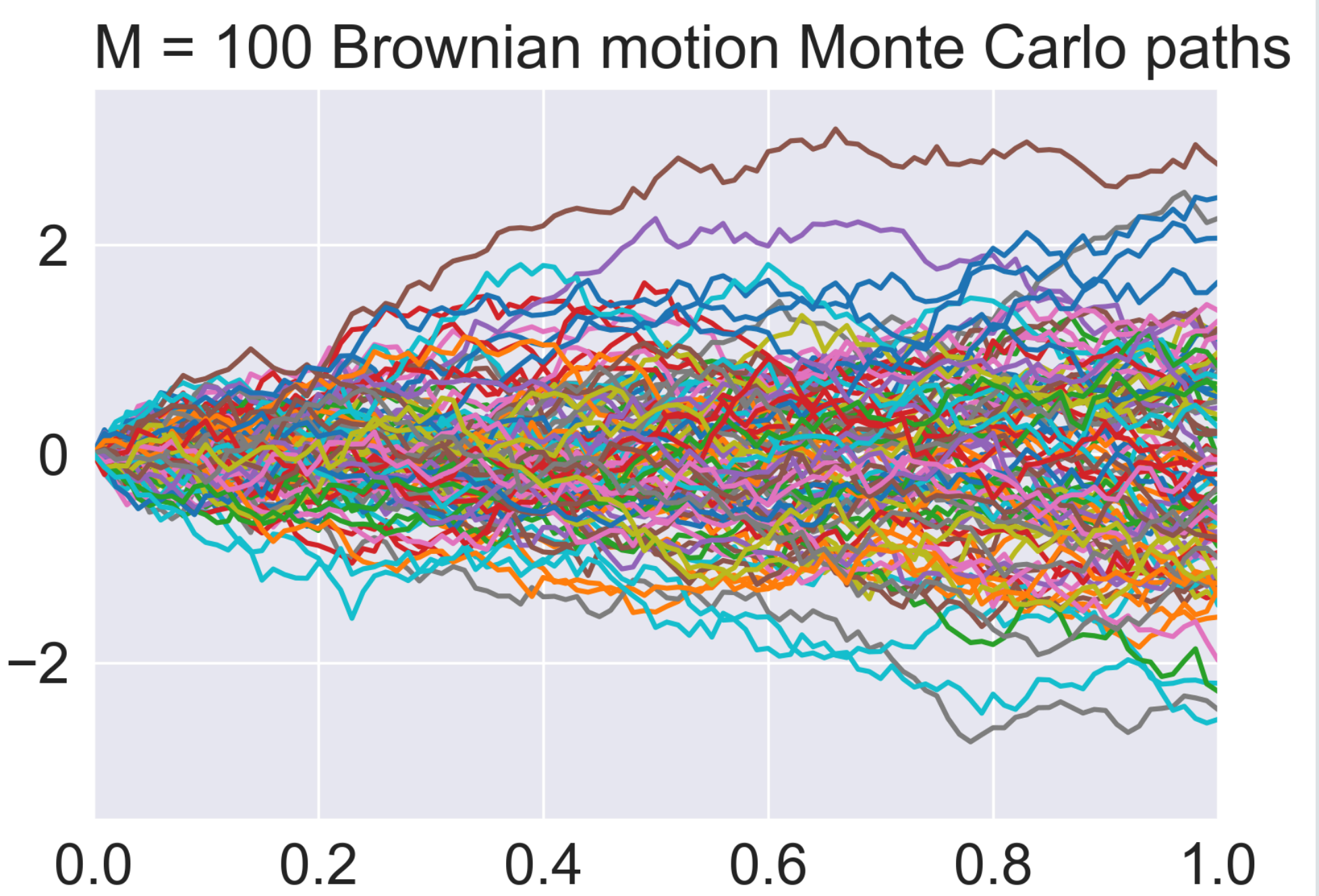
Simulating multiple BM paths: vectorization

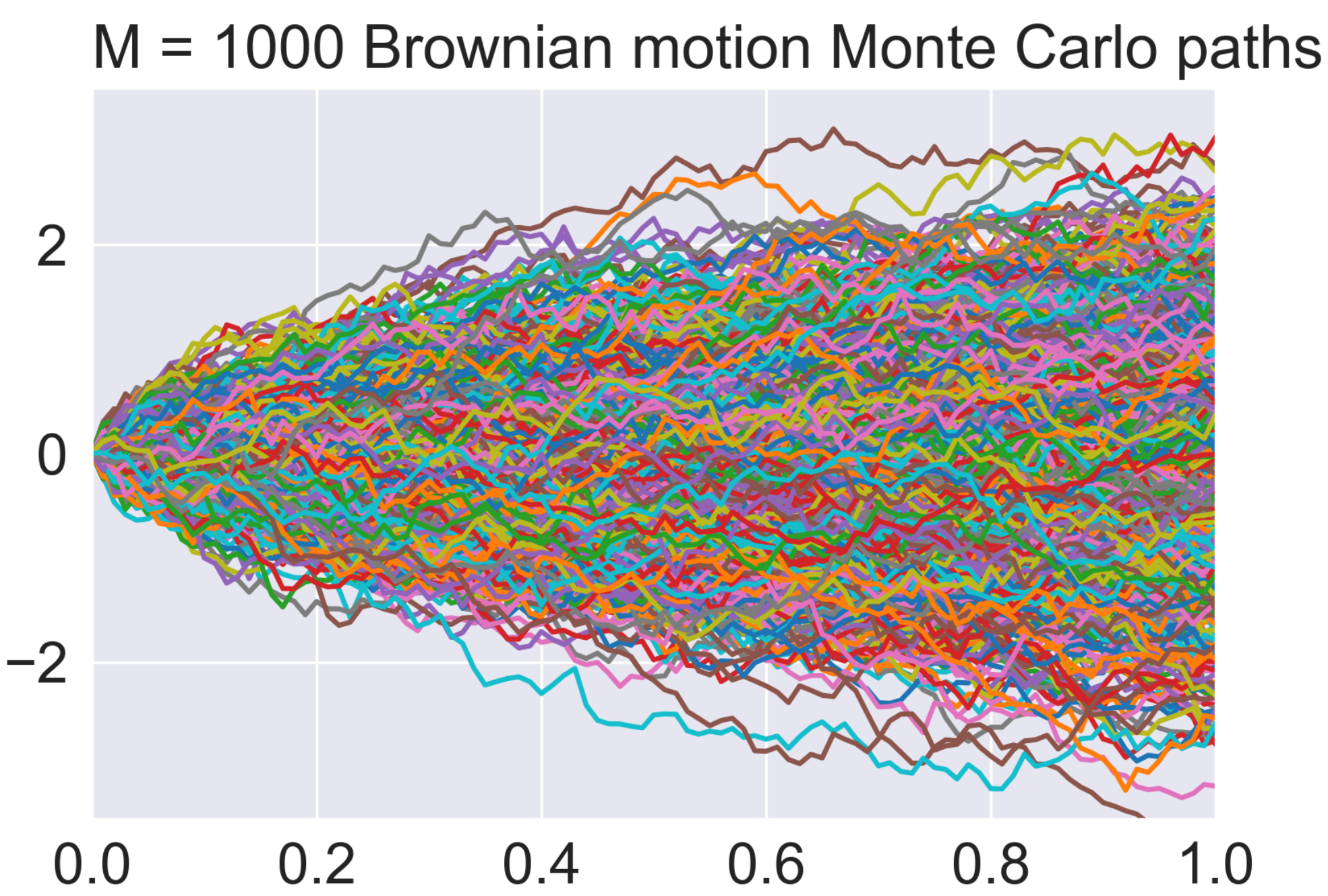
```
import time
import numpy as np
from scipy.stats import norm

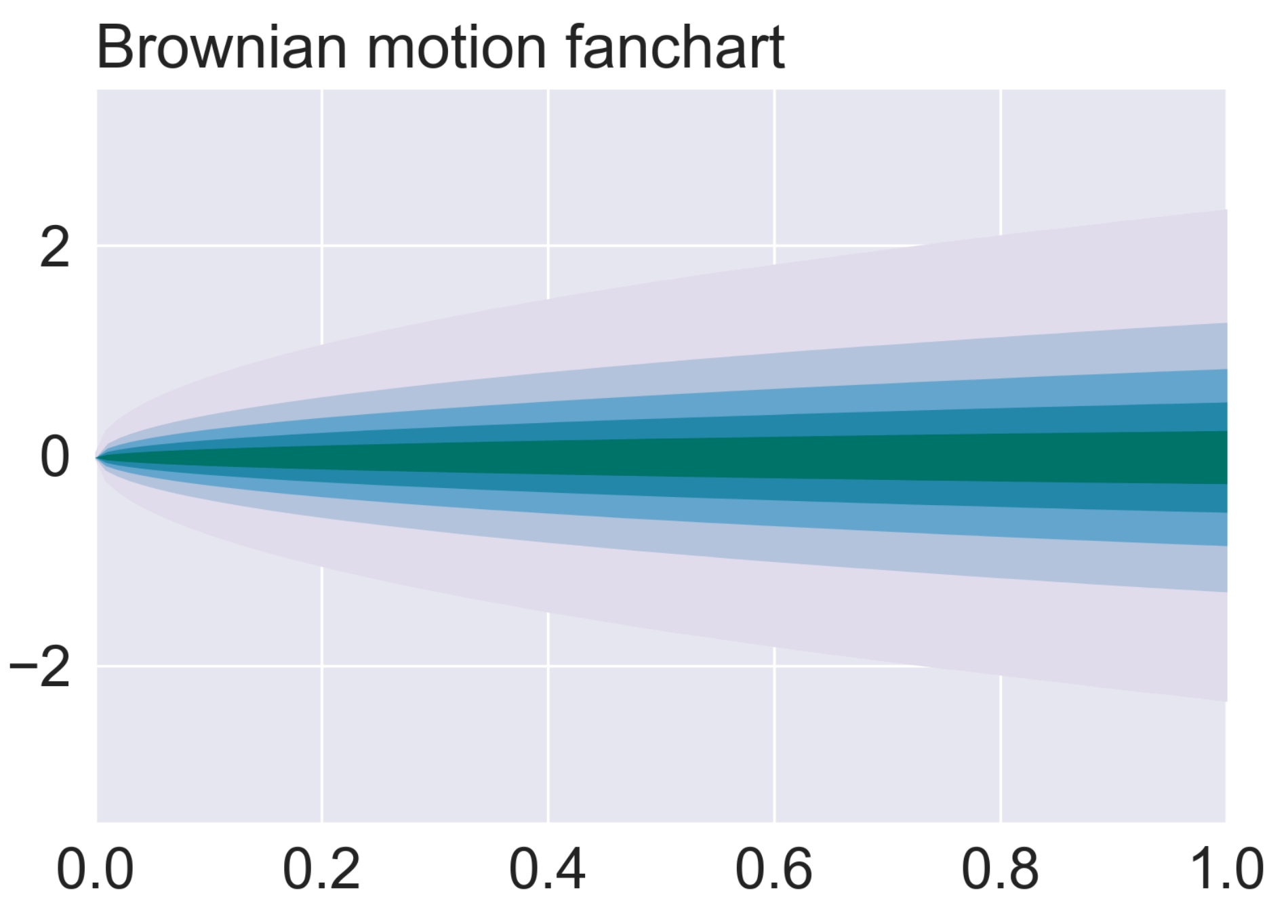
T = 1.0 # time horizon (option maturity)
N = 100 # number of time steps
M = 1000000 # number of Monte Carlo paths
dt = T/N # time step size
sqrtdt = np.sqrt(dt) # square root of time step
W = np.zeros([N+1,M]) # stores Brownian paths (row-major)
start_time = time.time()
for n in np.arange(N):
    # simulate all paths simultaneously
    W[n+1,:] = W[n,:] + sqrtdt*norm.rvs(size=M)
print('Execution time in seconds: ' + str(time.time() - start_time))
## Execution time in seconds: 4.675226926803589
```


M = 1 Brownian motion Monte Carlo path
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0


M = 10 Brownian motion Monte Carlo paths
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0

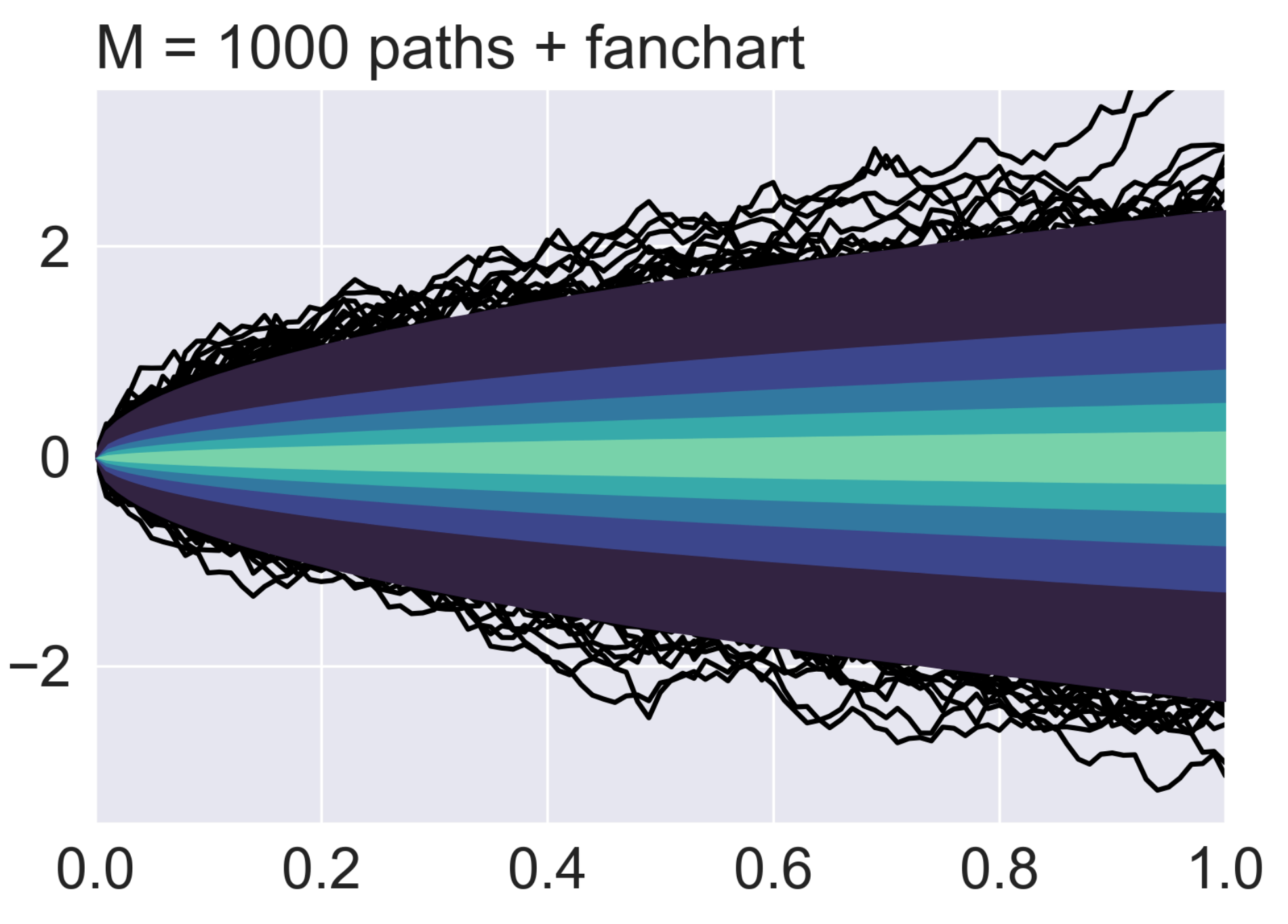

M = 100 Brownian motion Monte Carlo paths
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0


M = 1000 Brownian motion Monte Carlo paths
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0

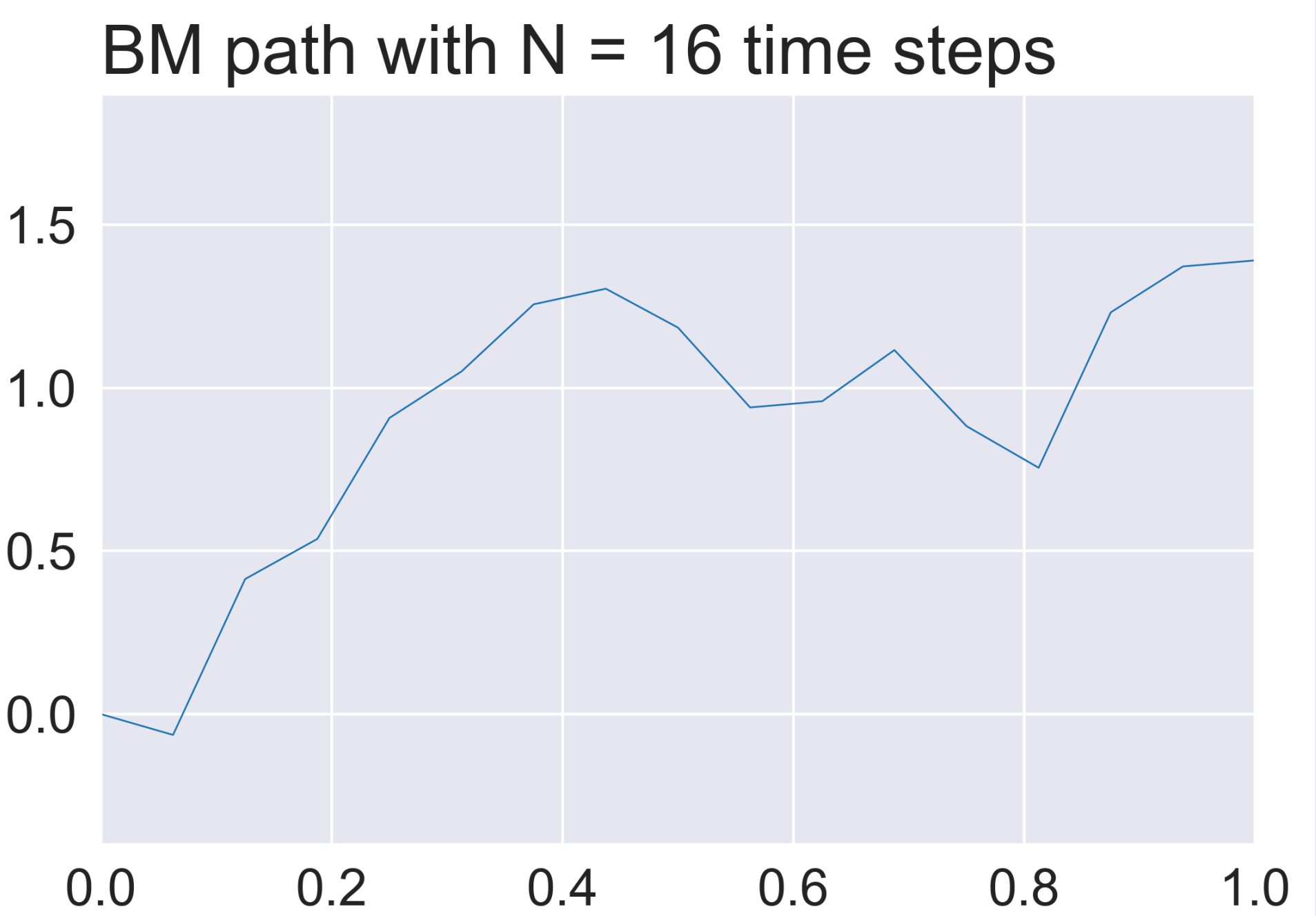

Brownian motion fanchart
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0

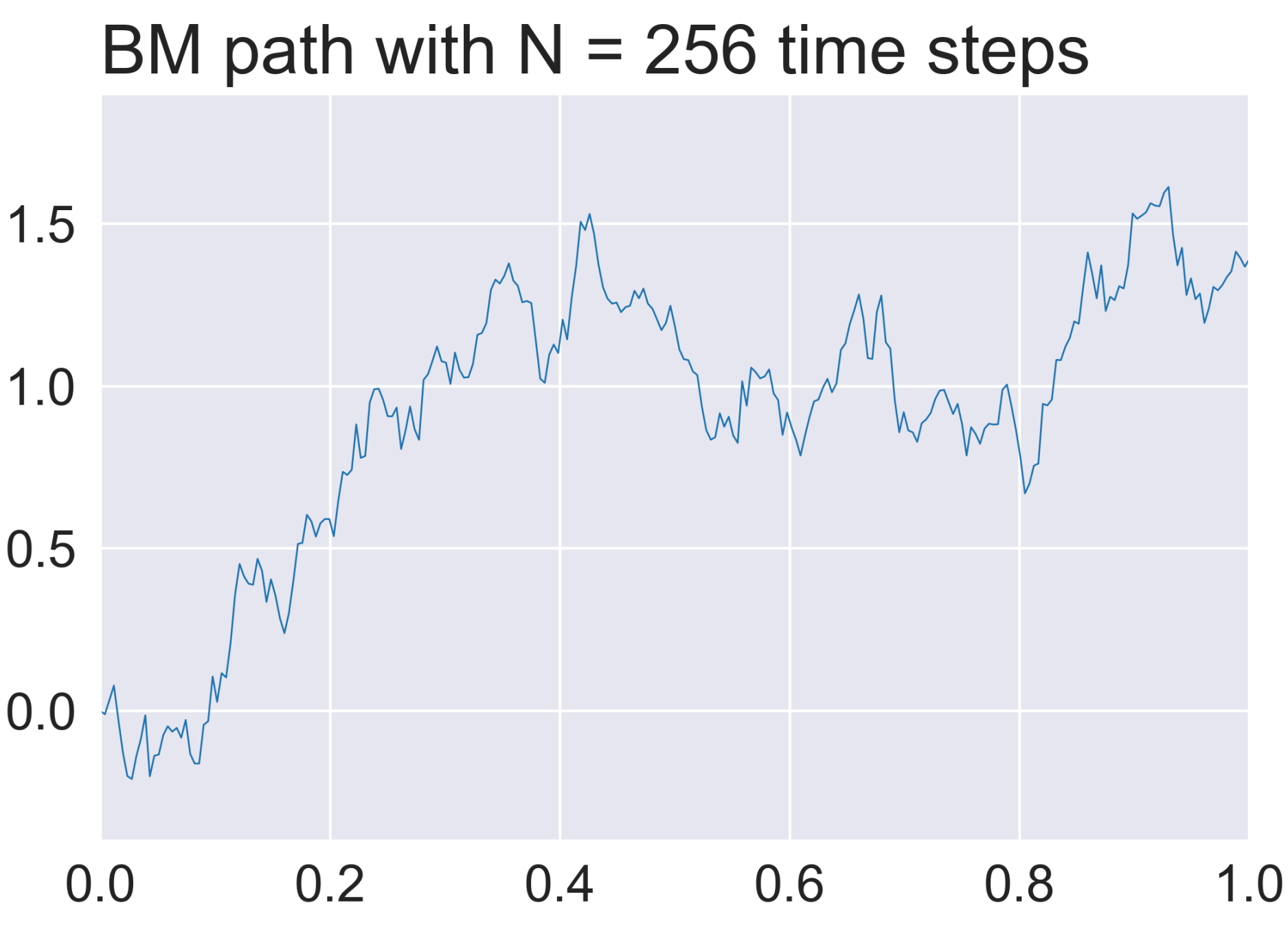
Fan chart: quantile line plots

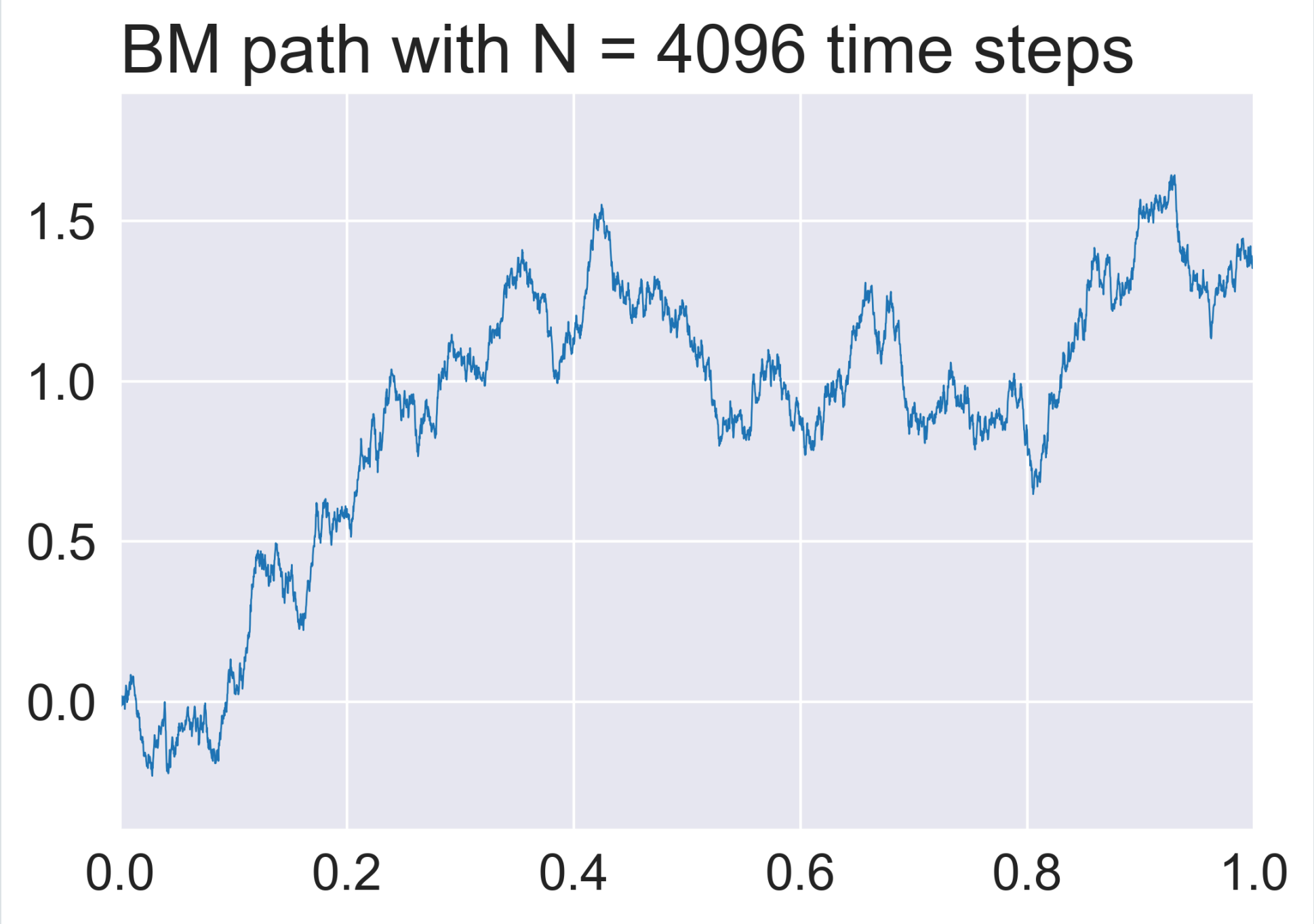
```
import seaborn as sns
n_band = 10 #number of color bands (the central two are merged)
col = sns.color_palette("PuBuGn_r",n_band//2)
q_val = np.linspace(0.0,1.0,n_band+1) # quantile levels
q_val[0] = 0.01
q_val[-1] = 0.99
Q = np.quantile(W,q_val,axis=1) # quantiles of BM paths for each time
fig, ax = plt.subplots(1)
for i in np.arange(n_band):
    col_idx = np.abs(int(i-n_band//2 + 0.5))
    ax.fill_between(tgrid,Q[i+1,:],Q[i,:],color=col[col_idx],zorder=20)
ax.set_xlim([0,T])
ax.set_ylim([-3.5,3.5])
plt.title("Brownian motion fanchart",loc="left")
plt.subplots_adjust()
plt.show()
```

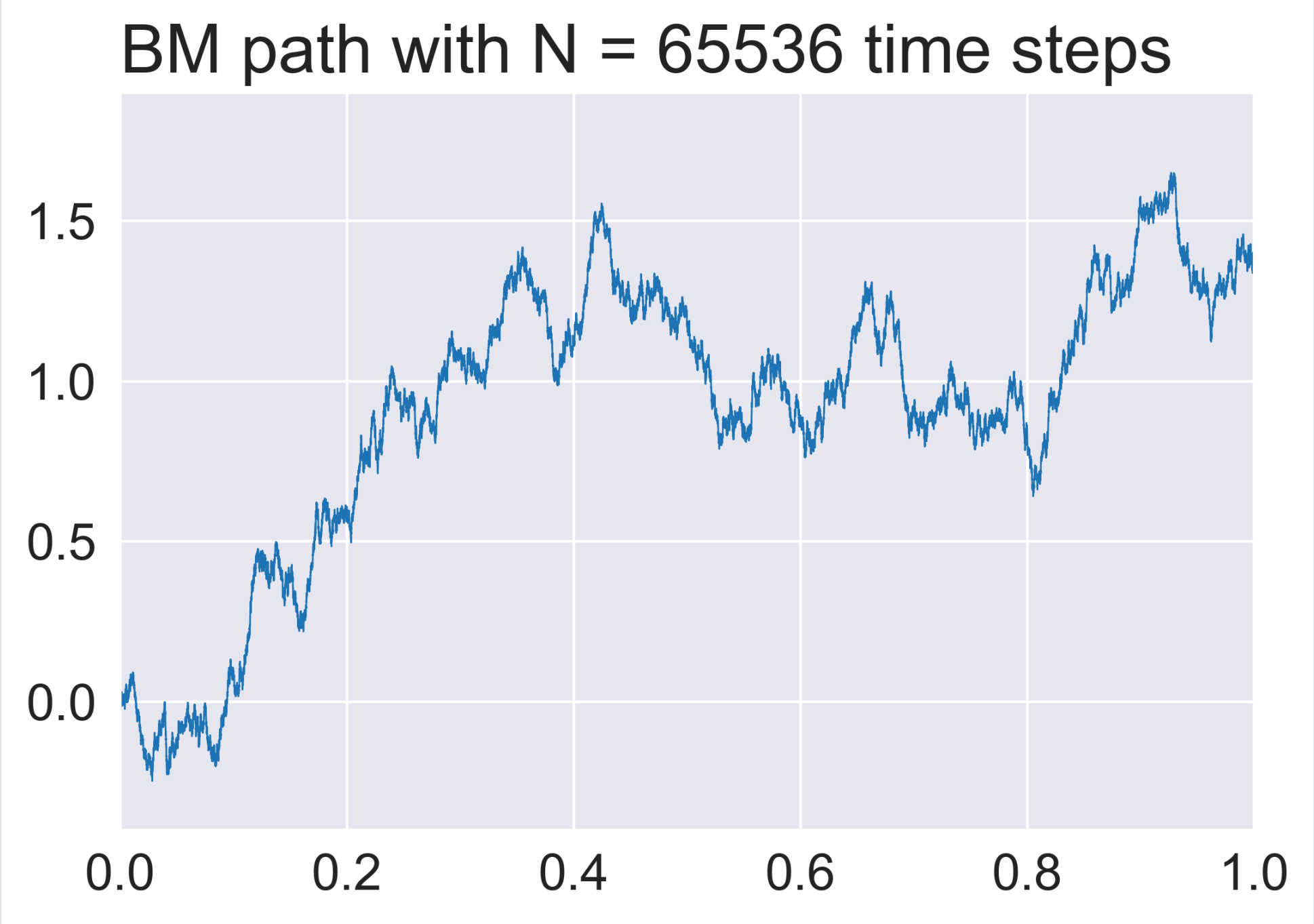

M = 1000 paths + fanchart
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0

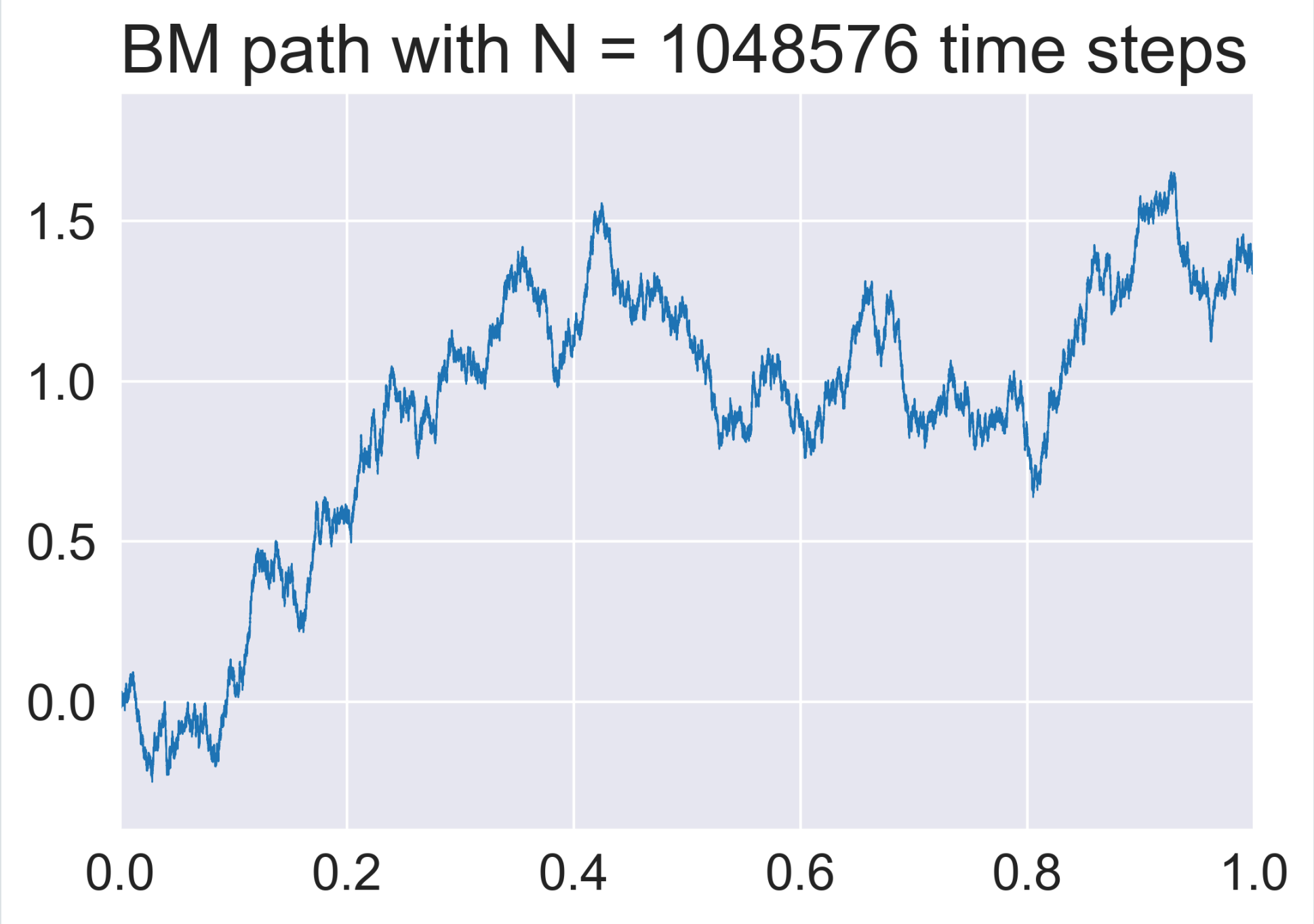
```
import seaborn as sns
import matplotlib.pyplot as plt
M = 1000 # number of BM paths displayed
n_band = 10 #number of color bands of the fan chart
col = sns.color_palette("mako_r",n_band//2)
q_val = np.linspace(0.0,1.0,n_band+1) # quantile levels
q_val[0] = 0.01
q_val[-1] = 0.99
Q = np.quantile(W,q_val,axis=1) # quantiles of BM paths for each time
tgrid = np.linspace(0.0,T,N+1)
fig, ax = plt.subplots()
ax.plot(tgrid,W,zorder=1,color="black") # BM paths
for i in np.arange(n_band):
    col_idx = np.abs(int(i-n_band//2 + 0.5))
    ax.fill_between(tgrid,Q[i+1,:],Q[i,:],color=col[col_idx],zorder=20)
ax.set_xlim([0,T])
ax.set_ylim([-3.5,3.5])
plt.title("M = " + str(M) +" paths + fanchart",loc="left")
```


BM path with N = 16 time steps
1.5
1.0
0.5
0.0
0.0
0.2
0.4
0.6
0.8
1.0


BM path with N = 256 time steps
1.5
1.0
0.5
0.0
0.0
0.2
0.4
0.6
0.8
1.0


BM path with N = 4096 time steps
1.5
1.0
0.5
0.0
0.0
0.2
0.4
0.6
0.8
1.0


BM path with N = 65536 time steps
1.5
1.0
0.5
0.0
0.0
0.2
0.4
0.6
0.8
1.0


BM path with N = 1048576 time steps
1.5
1.0
0.5
0.0
0.0
0.2
0.4
0.6
0.8
1.0

Simulate an arithmetic Brownian motion

An **arithmetic Brownian motion** $X = (X_t)_{0 \leq t \leq T}$ is the sum of a Brownian motion and a linear drift

$$X_t = X_0 + \mu t + \sigma W_t$$

where μ is a drift parameter and σ is a volatility parameter.
Simulating an arithmetic Brownian motion is simple:

- ▷ simulate the Brownian motion W as explained before
- ▷ multiply these paths by the constant σ
- ▷ add the affine function $X_0 + \mu t$ to each path

Geometric Brownian motion

A stochastic process $S = (S_t)_{0 \leq t \leq T}$ is said to follow a **geometric Brownian motion** (GBM) if it satisfies the following **stochastic differential equation** (SDE)

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

starting from a fixed value $S_0 > 0$, where μ is a drift parameter, σ is a volatility parameter and $W = (W_t)_{0 \leq t \leq T}$ is a Brownian motion. This SDE is equivalent to the following one, which uses Itô stochastic integrals:

$$S_t = S_0 + \int_0^t \mu S_u du + \int_0^t \sigma S_u dW_u$$

Geometric Brownian motion

This SDE is known to admit the following strong solution

$$S_t = S_0 \exp\left(\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma W_t\right)$$

This can be shown by applying Itô's lemma to $f(S_t)$ where $f = \log$:

$$df(S_t) = f'(S_t)dSt + \frac{1}{2}f''(S_t)d\langle S\rangle_t$$

Geometric Brownian motion

Since S satisfies $dS_t = \mu S_t dt + \sigma S_t W_t$, its **quadratic variation** is given by $d\langle S \rangle_t = \sigma^2 S_t^2 dt$, and Itô's lemma with $f = \log$ gives

$$d\log(S_t) = \frac{1}{S_t}(\mu S_t dt + \sigma S_t dW_t) - \frac{1}{2S_t^2}(\sigma^2 S_t^2 dt) = \left(\mu - \frac{\sigma^2}{2}\right) dt + \sigma dW_t$$

Meaning that

$$\log(S_t) - \log(S_0) = \left(\mu - \frac{\sigma^2}{2}\right) t + \sigma W_t$$

which gives the desired formula

$$S_t = S_0 \exp\left(\left(\mu - \frac{\sigma^2}{2}\right) t + \sigma W_t\right)$$

Simulate a geometric Brownian motion

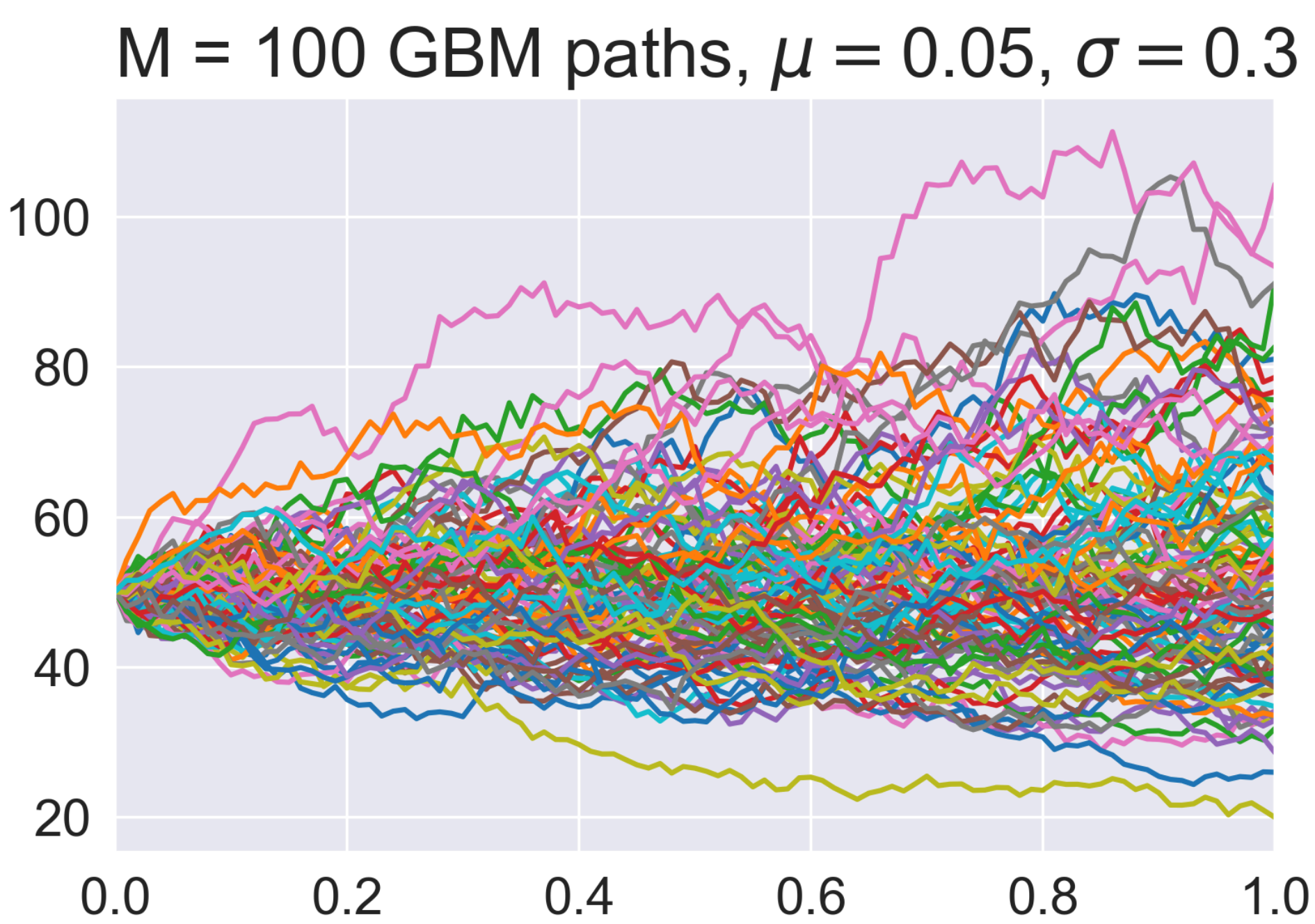
The best way to simulate a GBM is to take advantage of the strong solution of the GBM SDE:

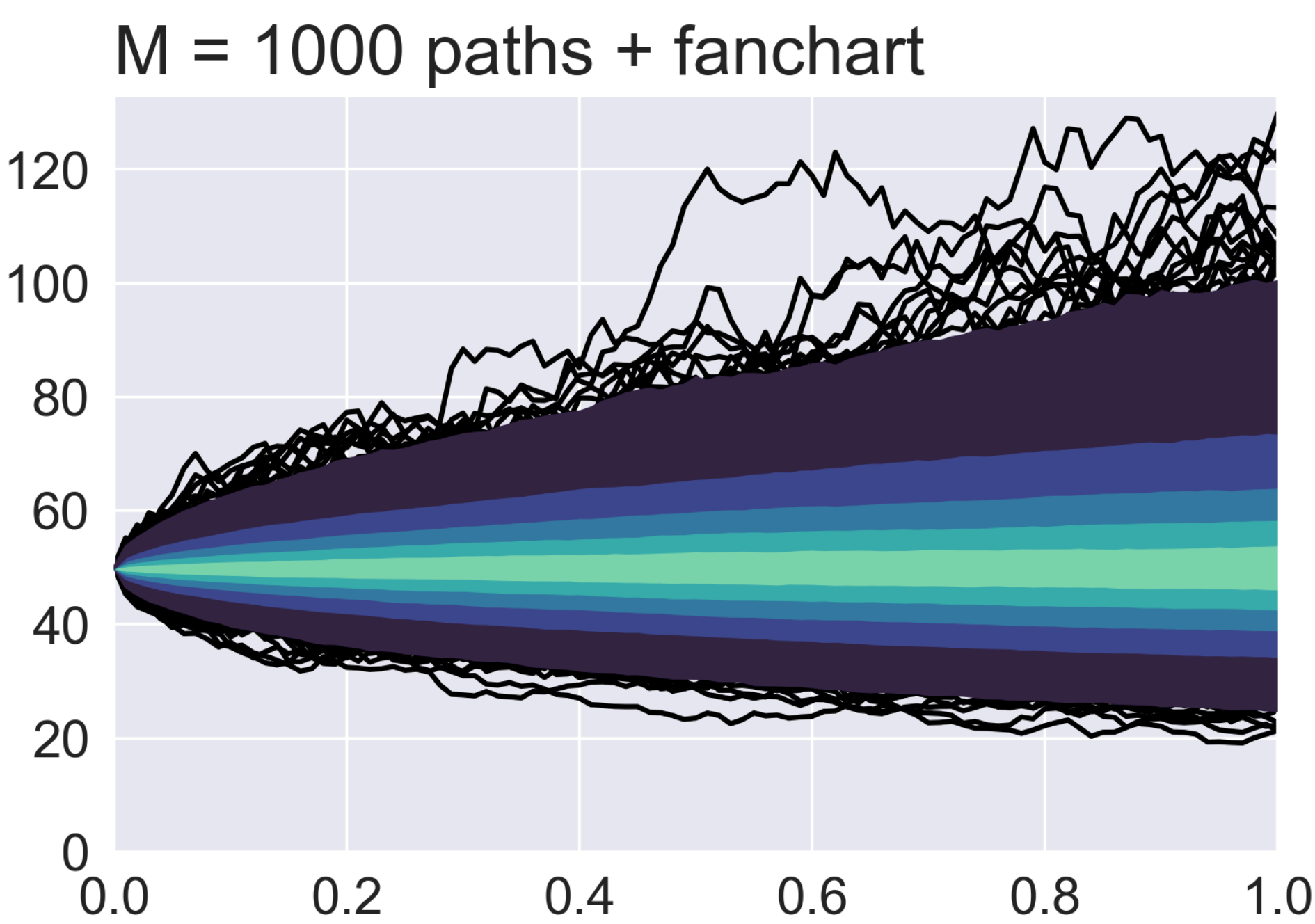
▷ simulate the arithmetic Brownian motion

$$\left(\mu - \frac{\sigma^2}{2}\right) t + \sigma W_t$$

as explained before

▷ Take the exponential of each path and multiply the results by S_0


M = 100 GBM paths, $\mu = 0.05$, $\sigma = 0.3$
100
80
60
40
20
0.0
0.2
0.4
0.6
0.8
1.0


M = 1000 paths + fanchart
120
100
80
60
40
20
0
0.0
0.2
0.4
0.6
0.8
1.0

General stochastic differential equation

Let the stochastic process $S = (S_t)_{0 \leq t \leq T}$ be the solution of the following general stochastic differential equation

$$dS_t = \mu(t, S_t)dt + \sigma(t, S_t)dW_t$$

starting from a fixed value $S_0 > 0$, where μ and σ are two real functions, and $W = (W_t)_{0 \leq t \leq T}$ is a Brownian motion. For the solution S to exist, the two functions μ and σ must satisfy some regularity conditions (for example Lipschitz-continuity).

In the general case, a general SDE of this type does not admit an explicit strong solution. We thus need to use a simulation algorithm which does not require the knowledge of the solution of the SDE.

Simulate an SDE: Euler scheme

The easiest and most popular way to simulate an SDE is the **Euler time-discretization scheme**. We use the same time grid as before $\mathcal{T} = \{t_0, t_1, \ldots, t_N\}$ such that $t_n = n \times \Delta_t$ where $\Delta_t := T/N$. The Euler scheme works as follows:

$$S_{t_{n+1}} = S_{t_n} + \mu(t_n, S_{t_n})\Delta_t + \sigma(t_n, S_{t_n})(W_{t_{n+1}} - W_{t_n})$$

starting from a fixed value $S_{t_0} = S_0$. To implement it in practice, we use the fact that $W_{t_{n+1}} - W_{t_n} \sim \mathcal{N}(0, \Delta_t)$:

$$S_{t_{n+1}} = S_{t_n} + \mu(t_n, S_{t_n})\Delta_t + \sigma(t_n, S_{t_n})\sqrt{\Delta_t}G_n$$

where G_n, $n = 1, \ldots, N$ are N independent standard Gaussian variables.

Time discretization error of Euler scheme

The Euler scheme provides an *approximation* of the (usually intractable) solution of the SDE

$$dS_t = \mu(t, S_t)dt + \sigma(t, S_t)dW_t$$

Under appropriate regularity conditions of μ and σ, the **discretization error** (which is a type of truncation error) between the solution SDE and the Euler approximation can be shown to be $O(\sqrt{\Delta_t})$.

The smaller the time step Δ_t, the smaller the discretization error of the Euler scheme. In other words, reducing the error between the solution of the SDE and its Euler approximation requires to set the number of time steps N as large as possible, which can quickly become impractical numerically.

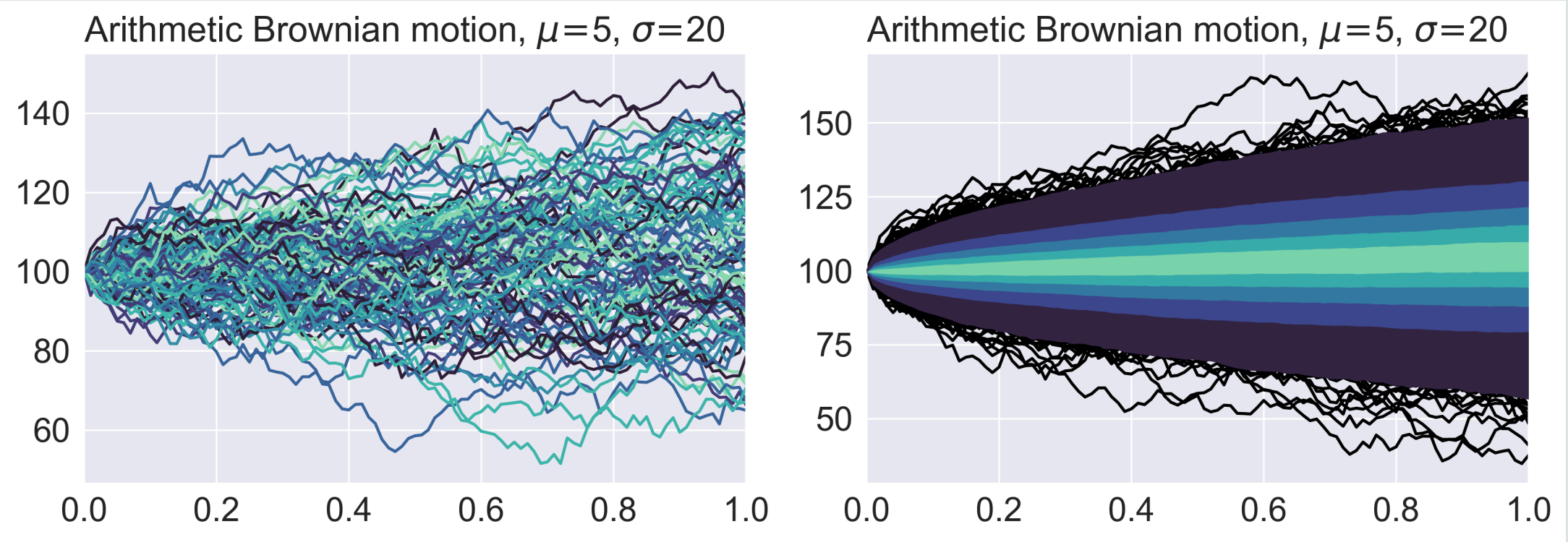
Time discretization error is not a critical issue

There are higher order schemes that reduce this discretization error. But we need to remember what our goals are. The SDE itself is *only a model*; modeling prices in continuous time is mathematically more convenient, but it is in itself an approximation of reality since prices do not change continuously. As a result, the time discretization "error" is not a concern at all for practitioners who favour the simple Euler scheme, so much so that it is possible to claim that **the Euler scheme IS the model**.

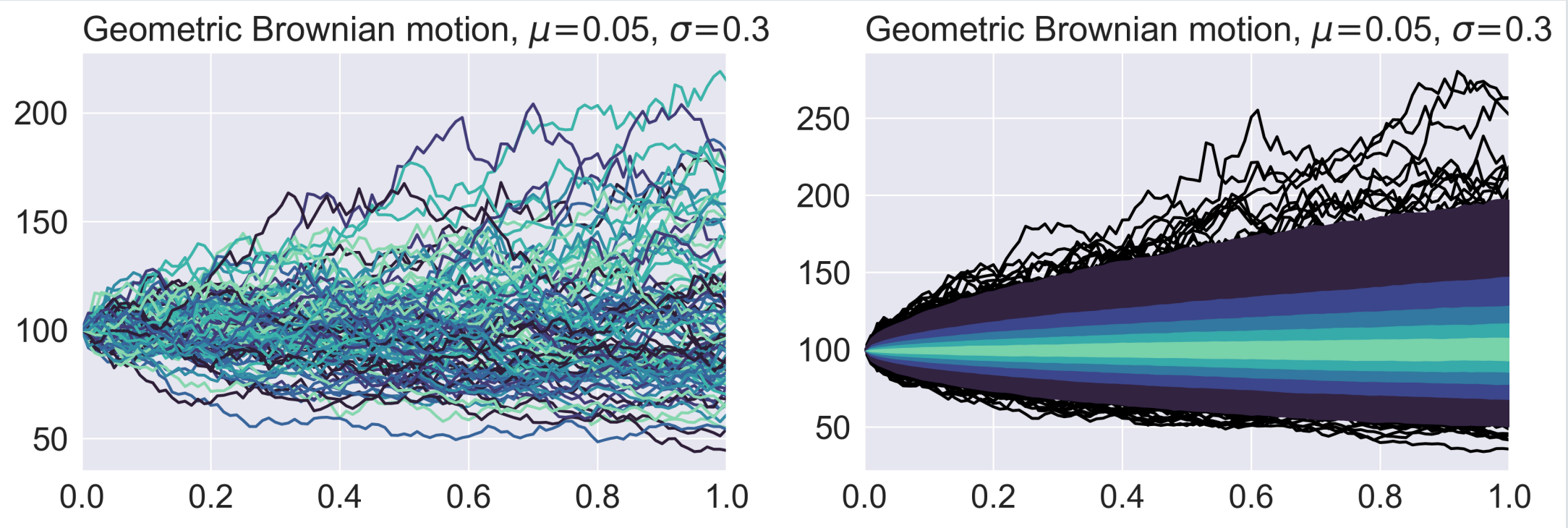
[1] P. Kloeden, E. Platen, and H. Schurz. *Numerical solution of SDE through computer experiments*. Springer Science & Business Media, 2012.

Stochastic process	SDE	$\mu(t,x)$	$\sigma(t,x)$
Brownian motion	$dS_t = dW_t$	0	1
arithmetic Brownian motion	$dS_t = \mu dt + \sigma dW_t$	μ	σ
geometric Brownian motion	$dS_t = \mu S_t dt + \sigma S_t dW_t$	μx	σx
Ornstein-Uhlenbeck process	$dS_t = \kappa(\mu - S_t)dt + \sigma dW_t$	$\kappa(\mu - x)$	σ
CIR process	$dS_t = \kappa(\mu - S_t)dt + \sigma\sqrt{S_t}dW_t$	$\kappa(\mu - x)$	$\sigma\sqrt{x}$
mean-reverting GBM / Inverse-Gamma process	$dS_t = \kappa(\mu - S_t)dt + \sigma S_t dW_t$	$\kappa(\mu - x)$	σx

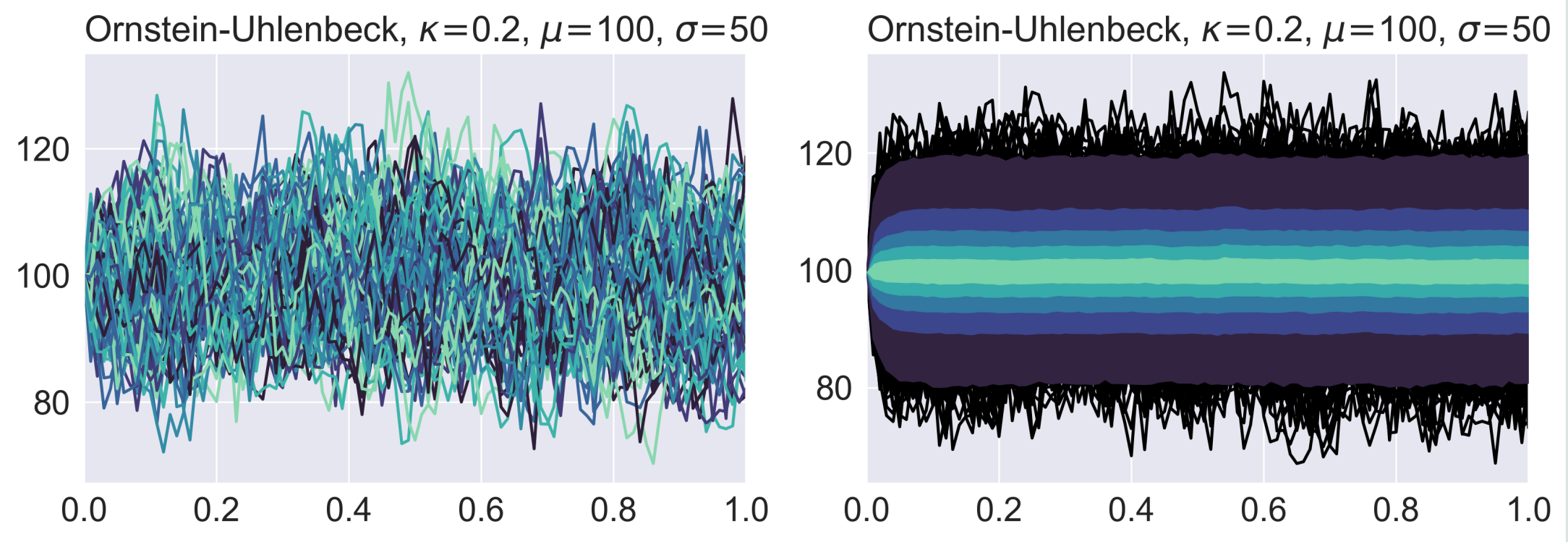
Arithmetic Brownian motion



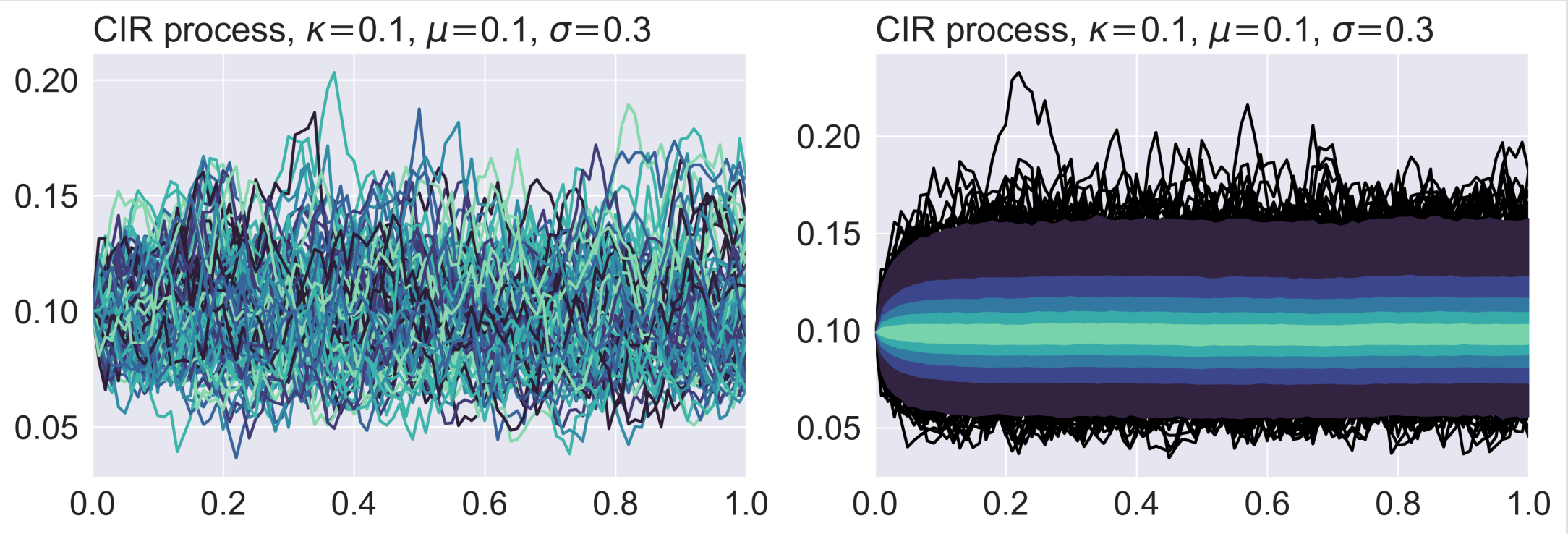
Geometric Brownian motion



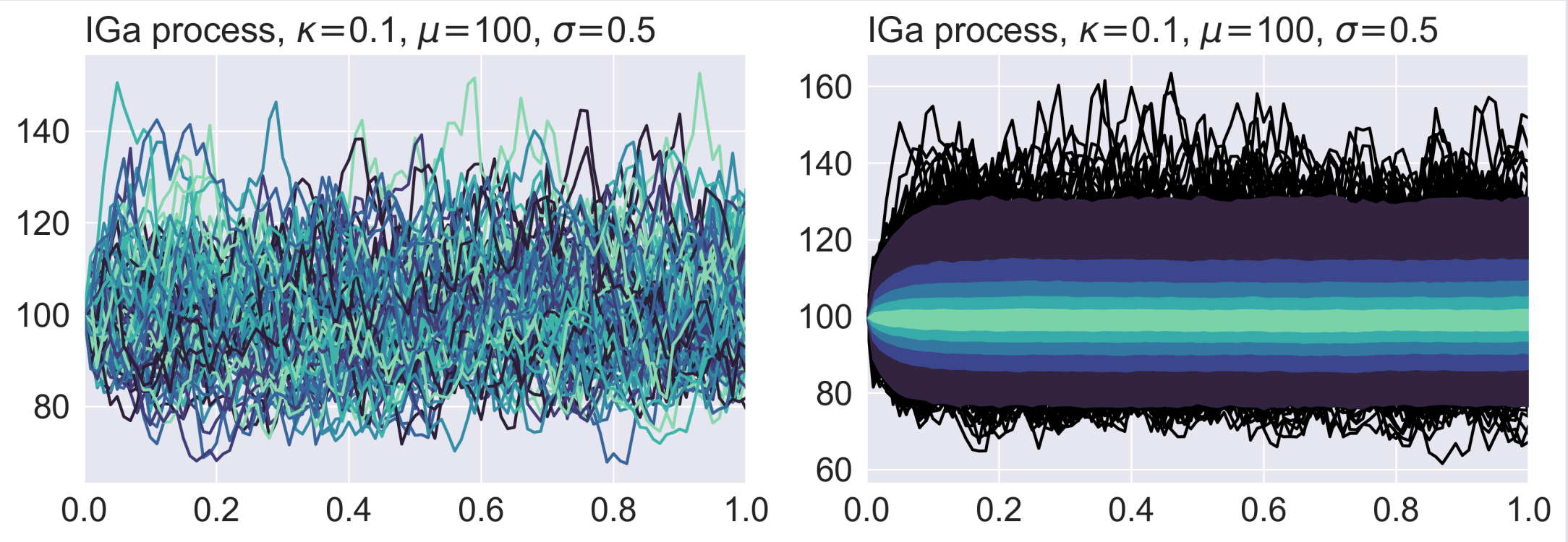
Ornstein-Uhlenbeck process



Cox-Ingersoll-Ross process



Inverse Gamma process



6.2 Simulation of correlated stochastic processes

Correlated Brownian motions

Let $W = (W_t)_{0 \leq t \leq T}$ and $B = (B_t)_{0 \leq t \leq T}$ be two standard Brownian motions. W and B are said to be **correlated Brownian motions** with constant correlation ρ if for any $0 \leq s < t$,

$$\mathbb{C}\text{orr}\left(W_t - W_s, B_t - B_s\right) = \rho$$

In particular, $\mathbb{C}\text{ov}\left(W_t - W_s, B_t - B_s\right) = \rho(t-s)$, and the random vector $(W_t - W_s, B_t - B_s)$ follows a bivariate Gaussian distribution:

$$\begin{bmatrix} W_t - W_s \\ B_t - B_s \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} t-s & \rho(t-s) \\ \rho(t-s) & t-s \end{bmatrix}\right)$$

Simulate correlated Brownian motions

Introduce the time grid $\mathcal{T} = \{t_0, t_1, \ldots, t_N\}$ such that $t_n = n \times \Delta_t$ where $\Delta_t := T/N$. Then:

▷ Simulate N independent draws (G_n, H_n), $n = 1, \ldots, N$ from a bivariate standard Gaussian vector

▷ Set $W_{t_0} = 0$, $B_{t_0} = 0$ and for every $n = 0, 1, \ldots, N-1$, define

$$W_{t_{n+1}} = W_{t_n} + \sqrt{\Delta_t} G_n$$

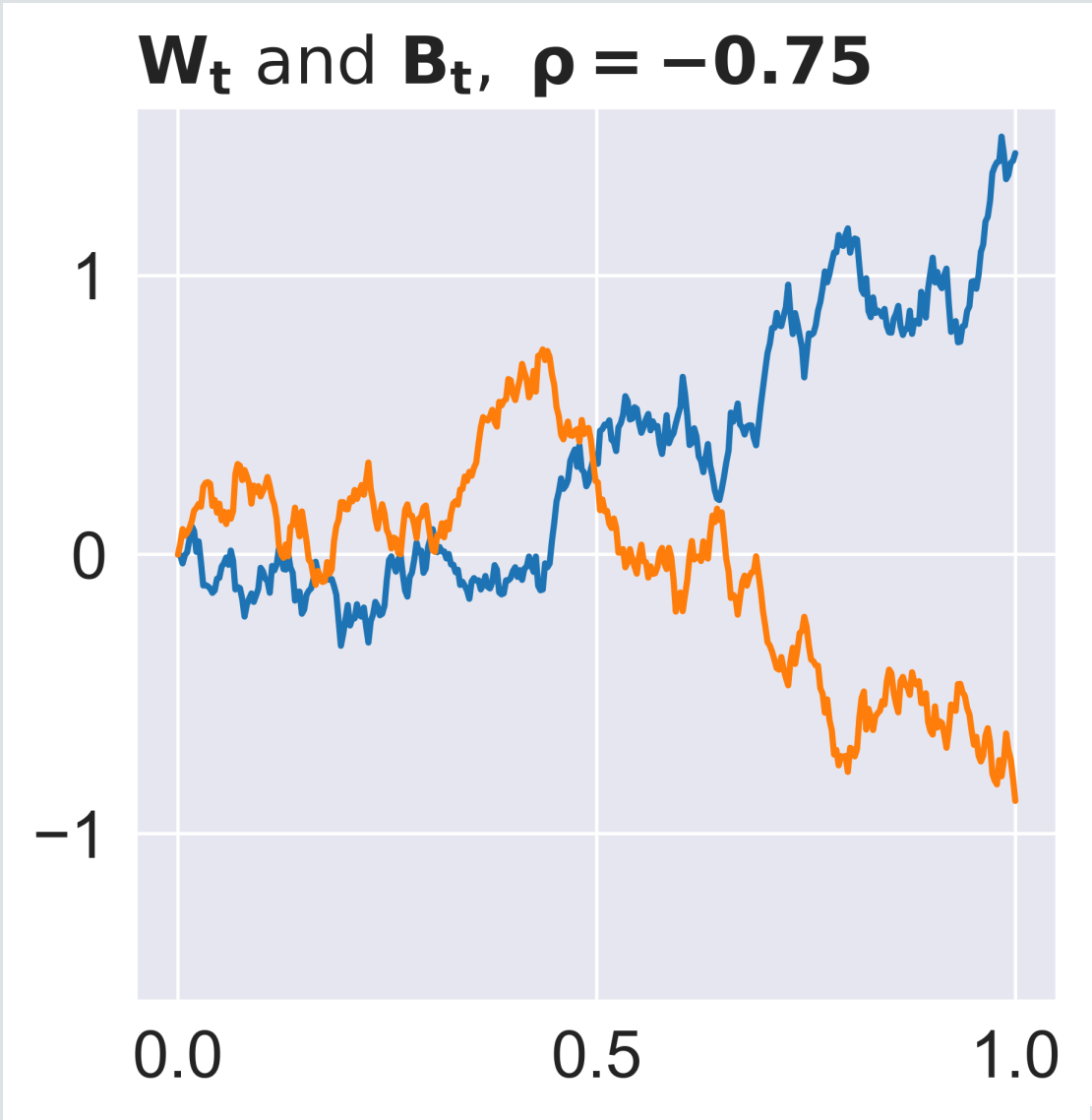
$$B_{t_{n+1}} = B_{t_n} + \sqrt{\Delta_t} \left(\rho G_n + \sqrt{1 - \rho^2} H_n \right)$$

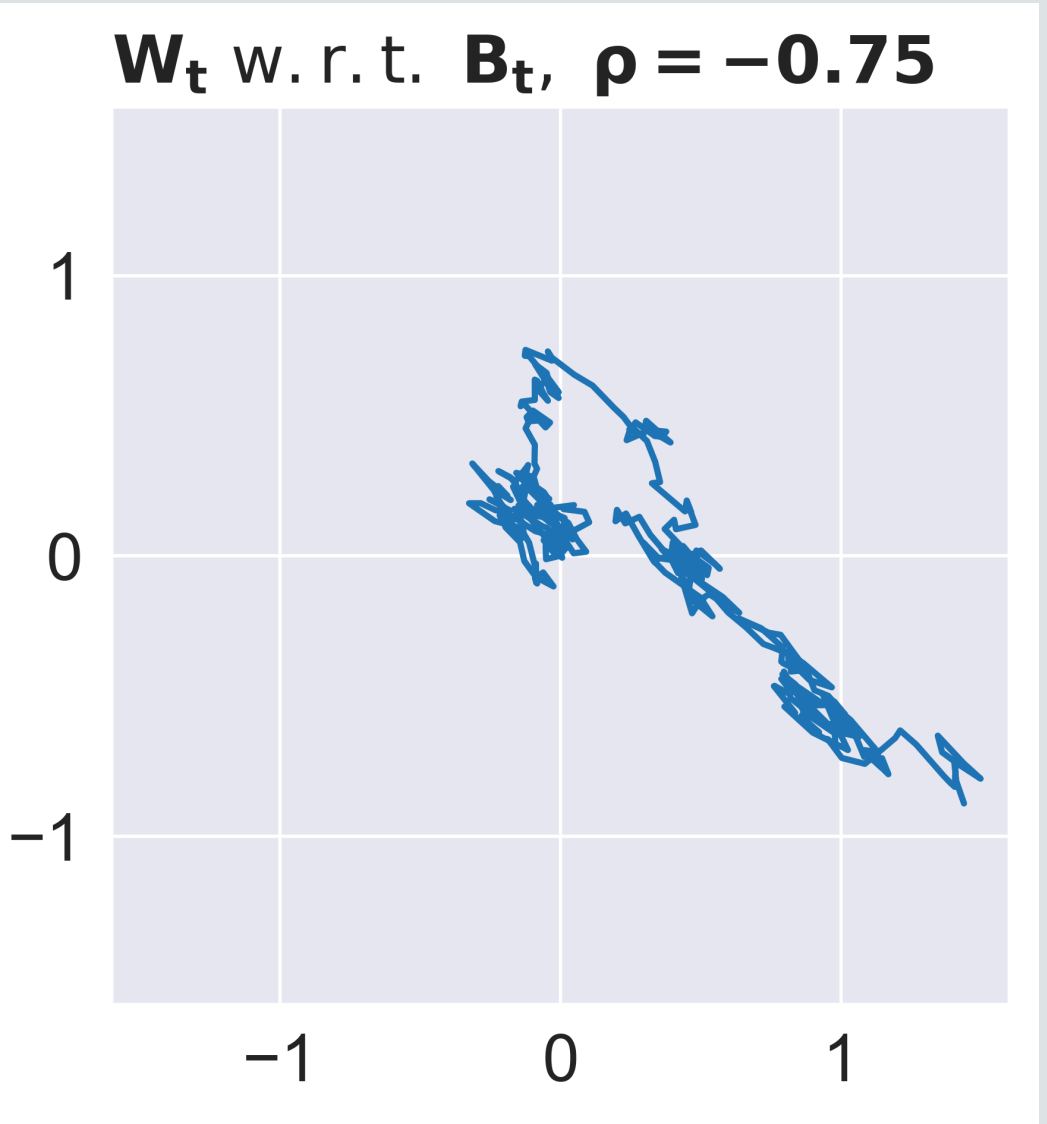
```
import numpy as np; from scipy.stats import norm
import matplotlib.pyplot as plt; import seaborn as sns
sns.set_style("darkgrid"); sns.set_palette("mako")

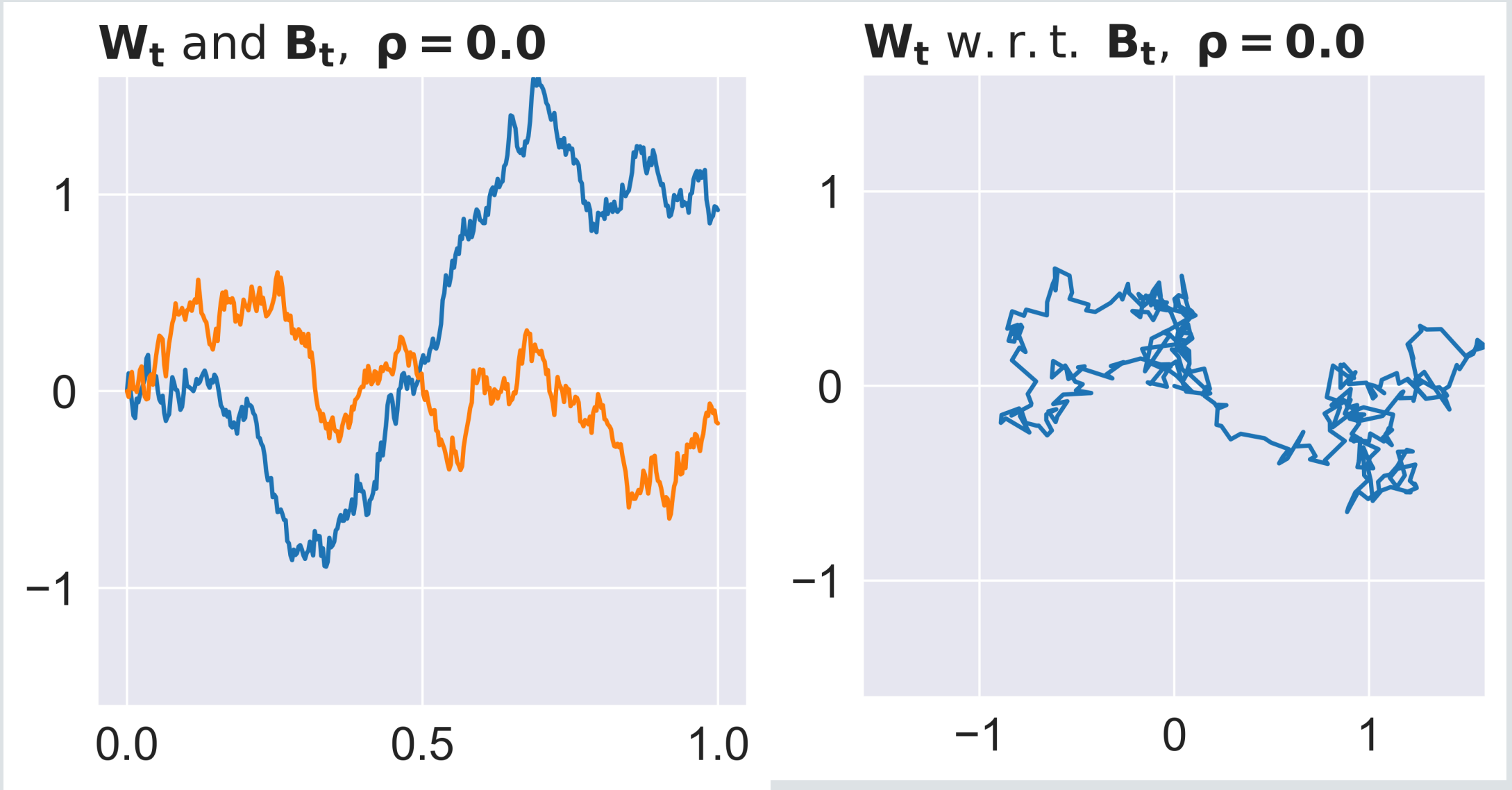
M = 25 # number of Monte Carlo paths
N = 3000 # number of time steps
T = 1.0 # time horizon (option maturity)
dt = T/N # time step size
sqrtdt = np.sqrt(dt) # square root of time step

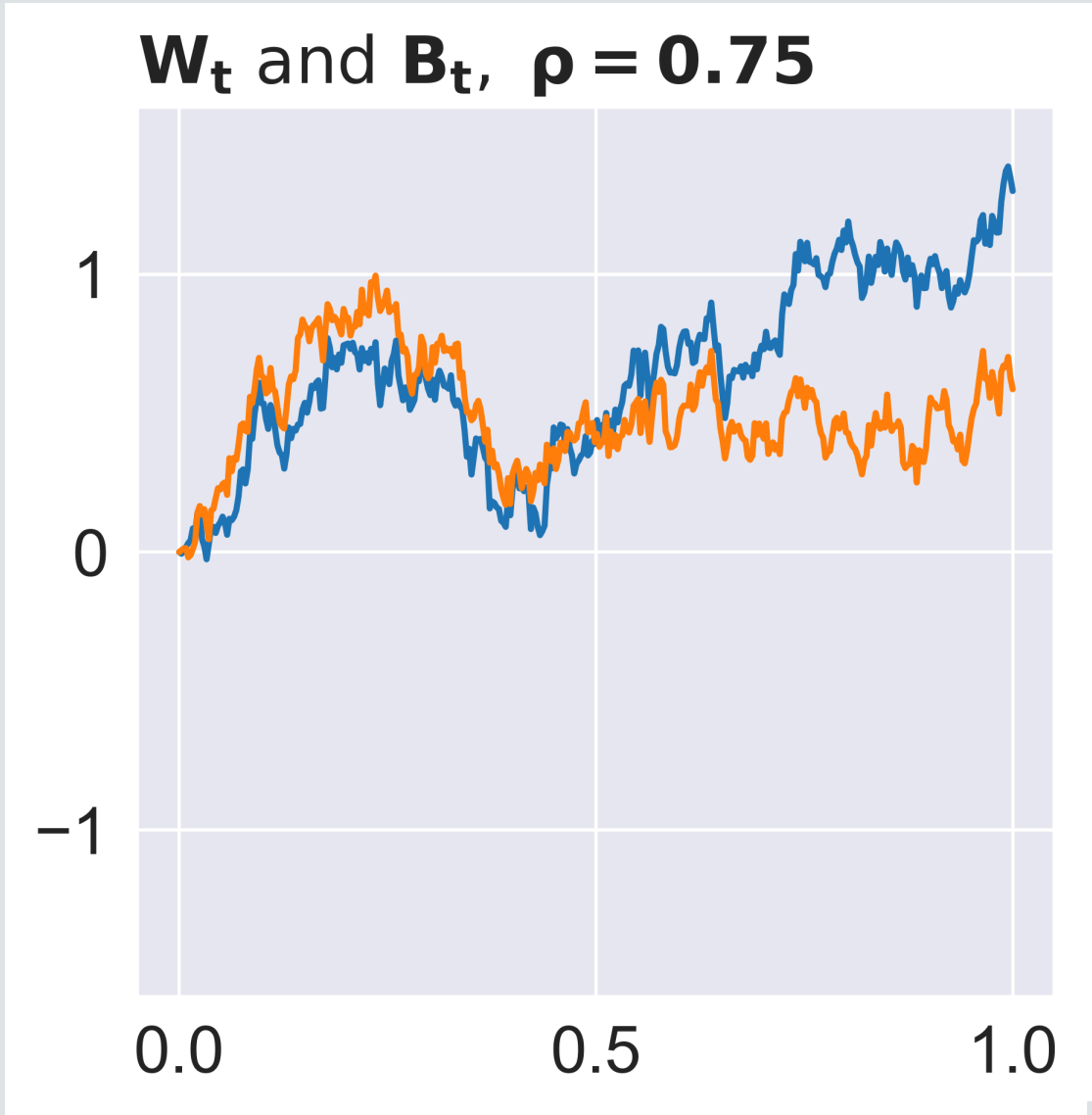
rho = 0.00 # change correlation value here
W = np.zeros([N+1,M])
B = np.zeros([N+1,M])
for n in np.arange(N):
    dW = sqrtdt*norm.rvs(size=M)
    dB = rho*dW + np.sqrt(1-rho*rho) * sqrtdt*norm.rvs(size=M)
    W[n+1,:] = W[n,:] + dW
    B[n+1,:] = B[n,:] + dB

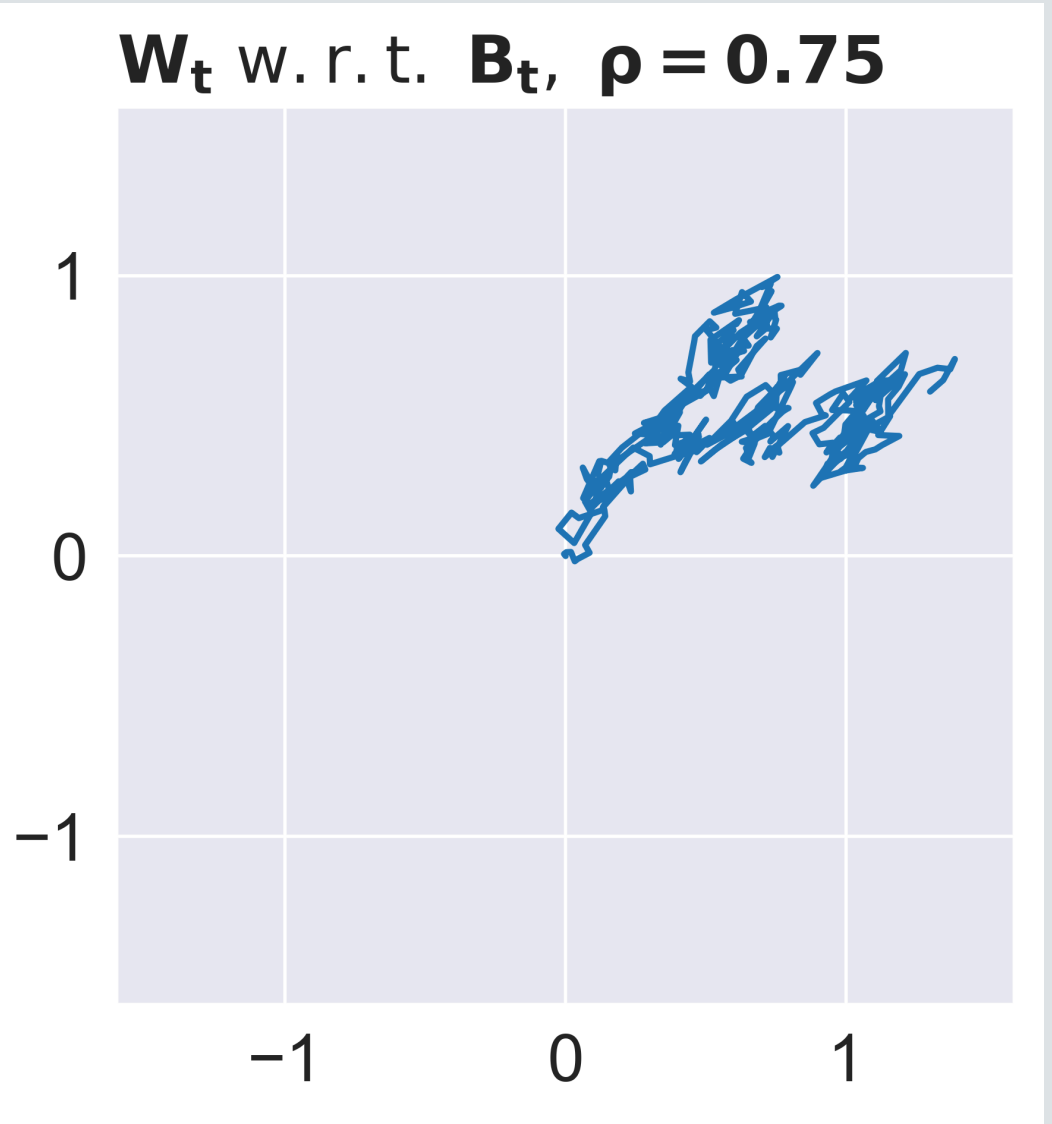
fig, ax = plt.subplots()
ax.plot(B,W,linewidth=0.8,alpha=0.9)
ax.set_aspect('equal', 'box')
plt.title(r'$\mathbf{\rho='+str(rho)+',\ M='+str(M)+'}$', loc='left')
```

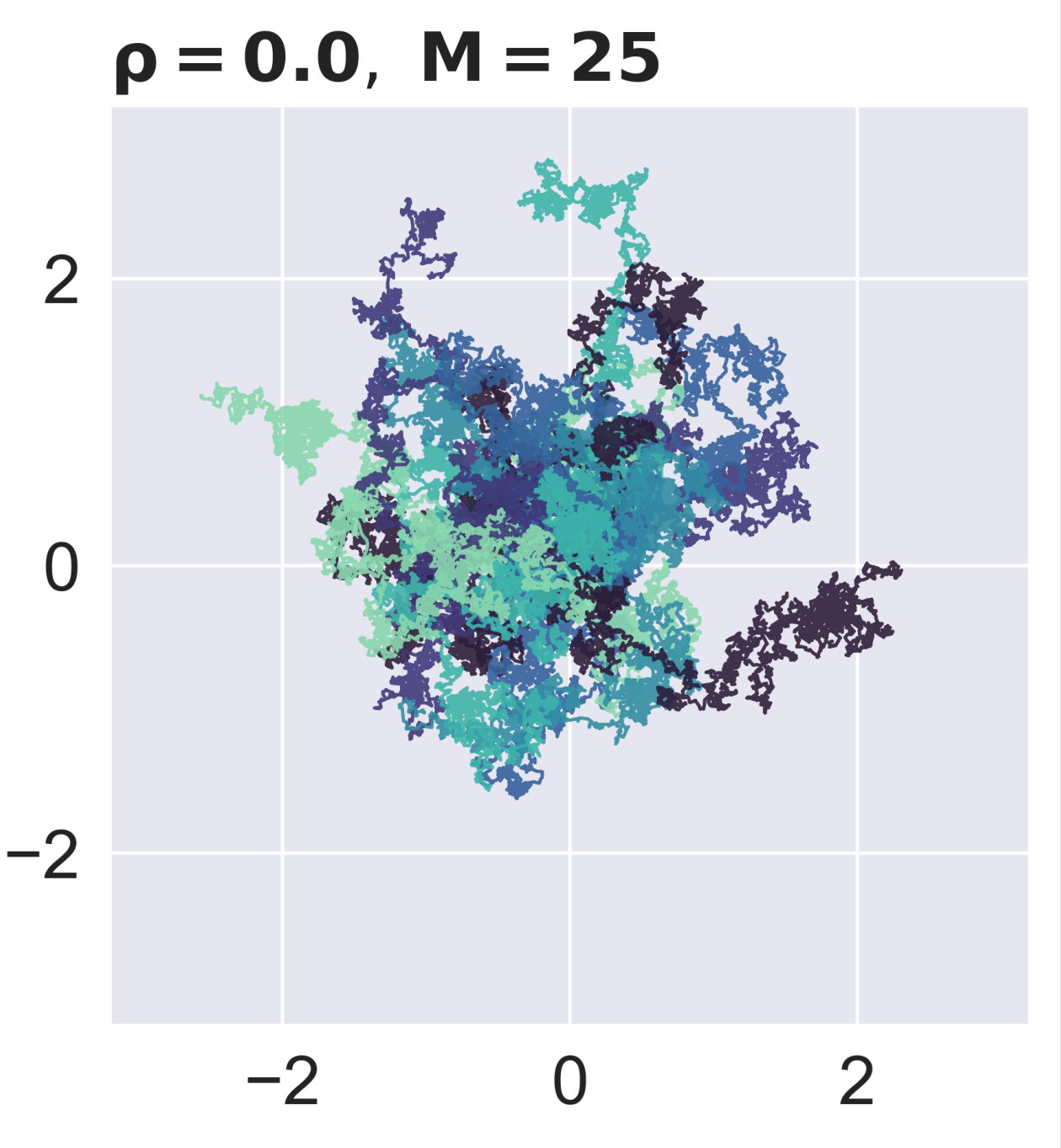

W_t and B_t, $\rho = -0.75$
1
0
−1
0.0
0.5
1.0

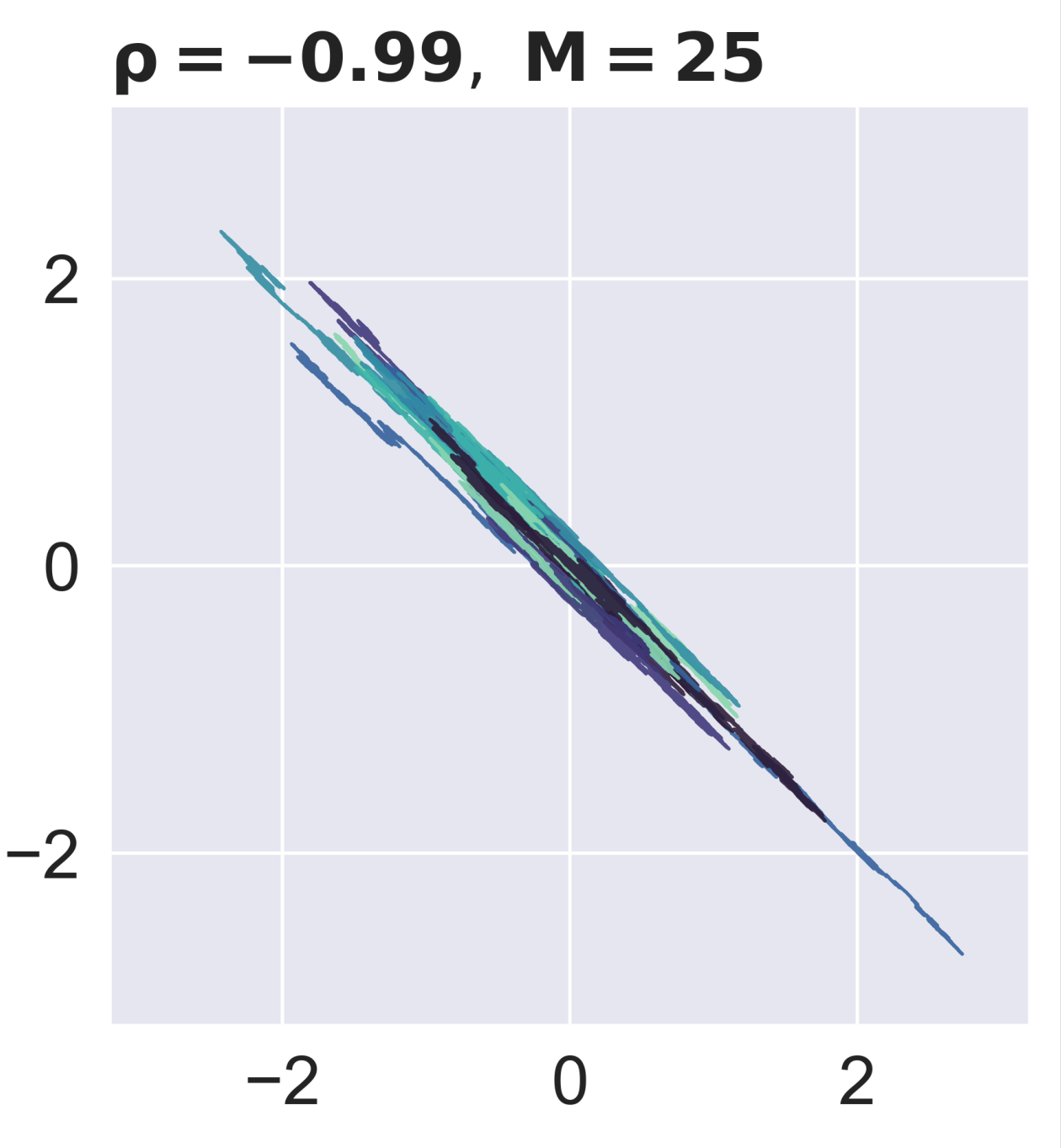

W_t w. r. t. B_t, $\rho = -0.75$
1
0
−1
−1
0
1

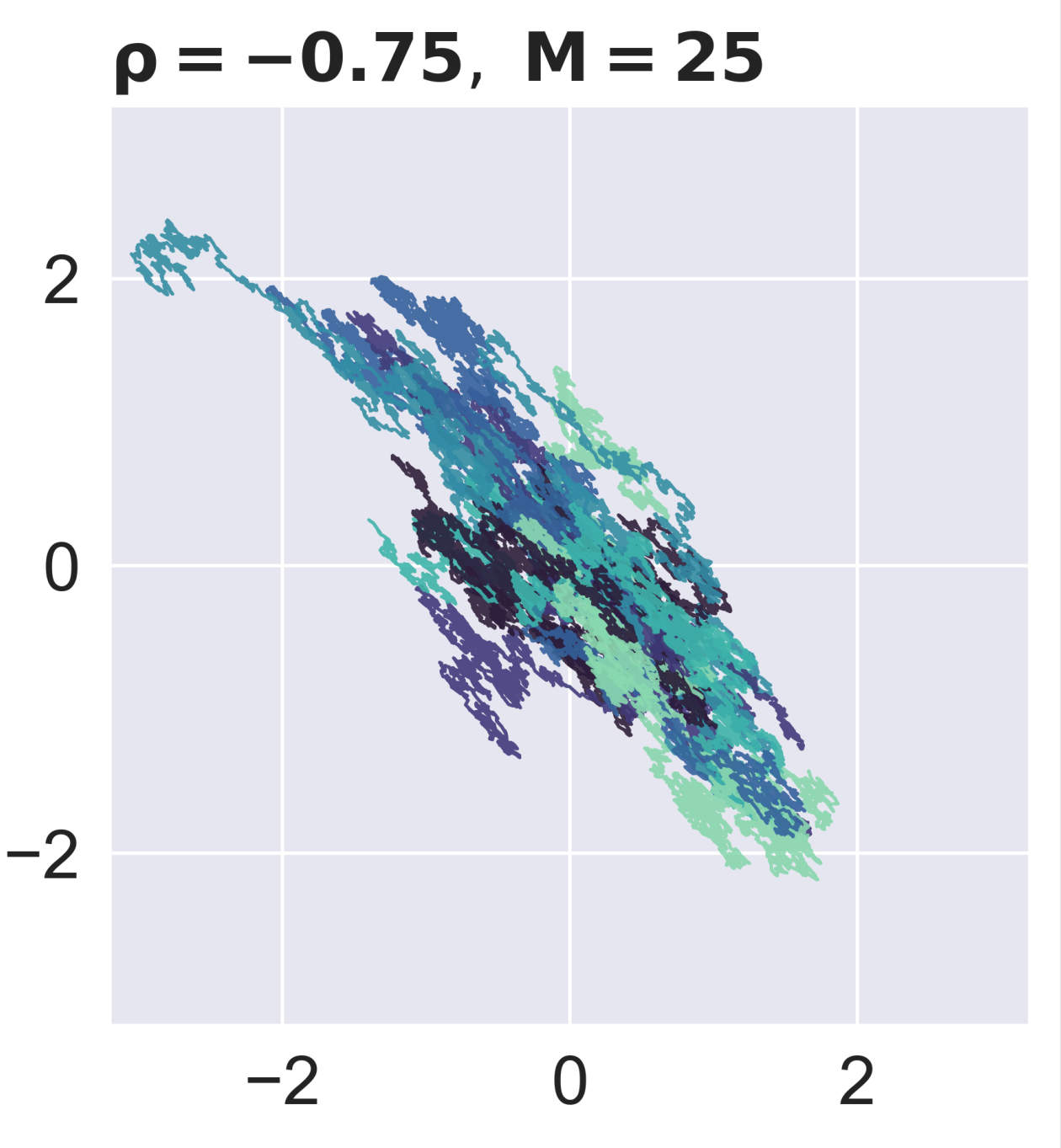

W_t and B_t, $\rho = 0.0$
1
0
−1
0.0
0.5
1.0
W_t w. r. t. B_t, $\rho = 0.0$
1
0
−1
−1
0
1

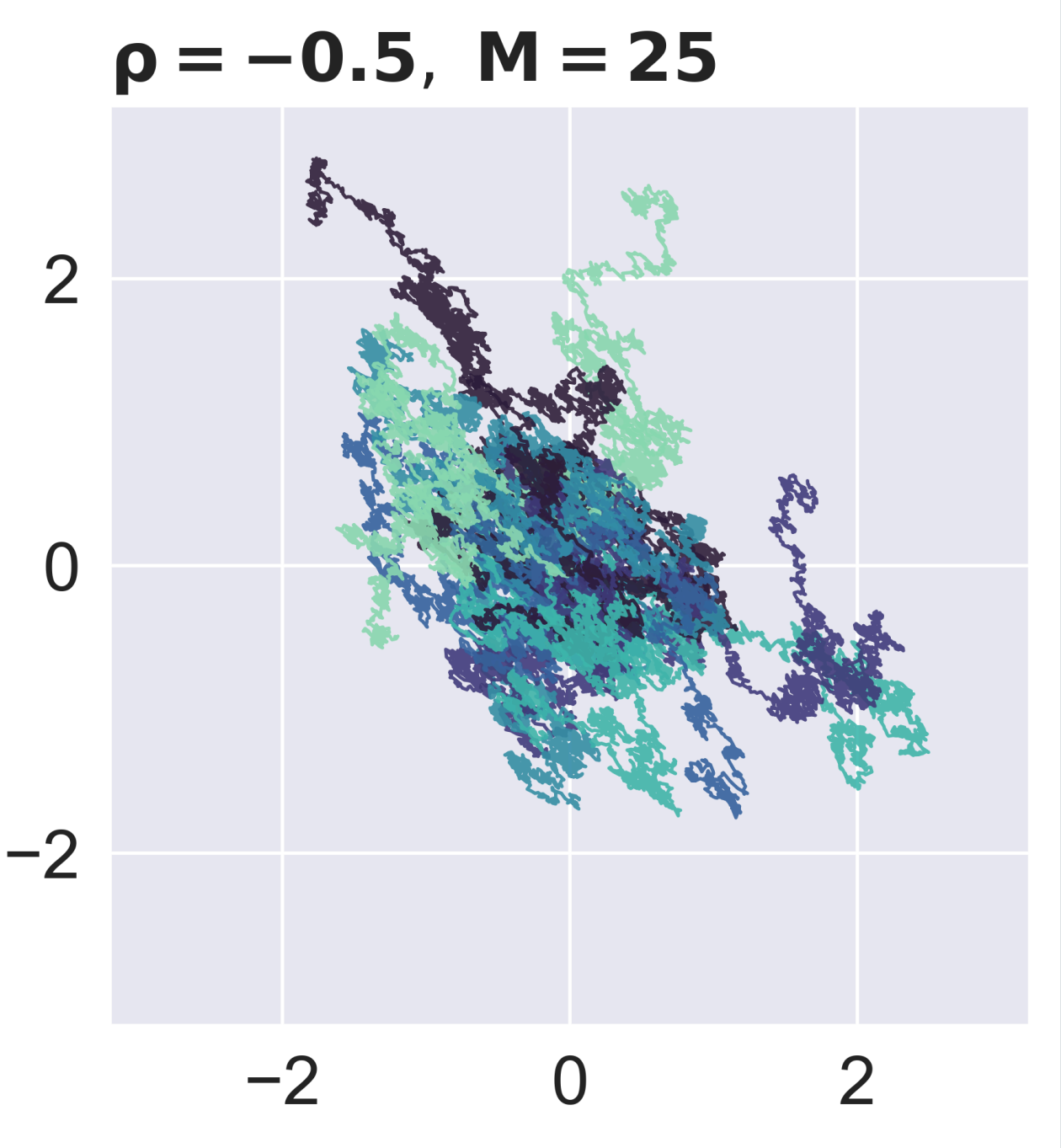

W_t and B_t, $\rho = 0.75$
1
0
−1
0.0
0.5
1.0

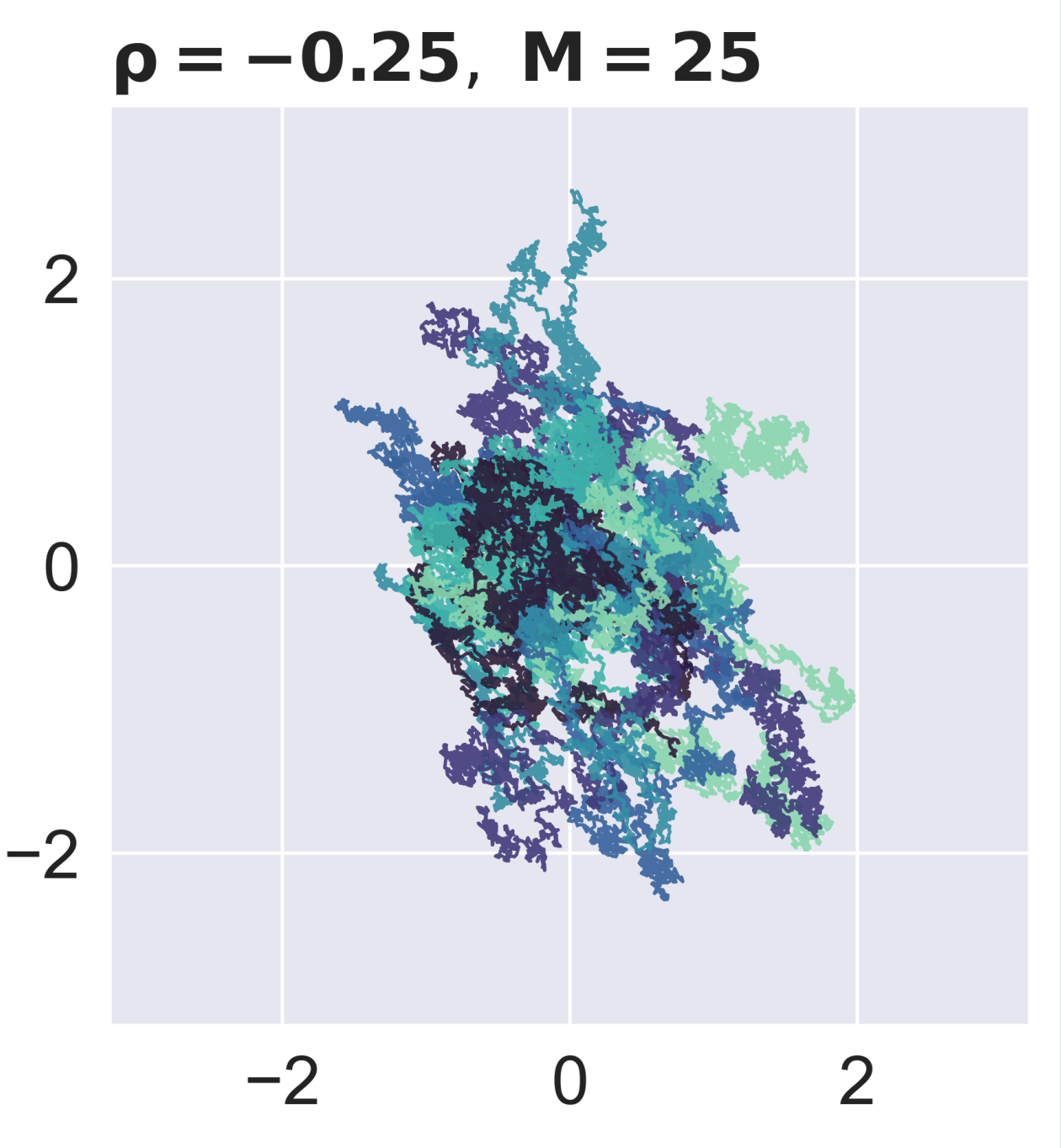

W_t w. r. t. B_t, $\rho = 0.75$
1
0
−1
−1
0
1

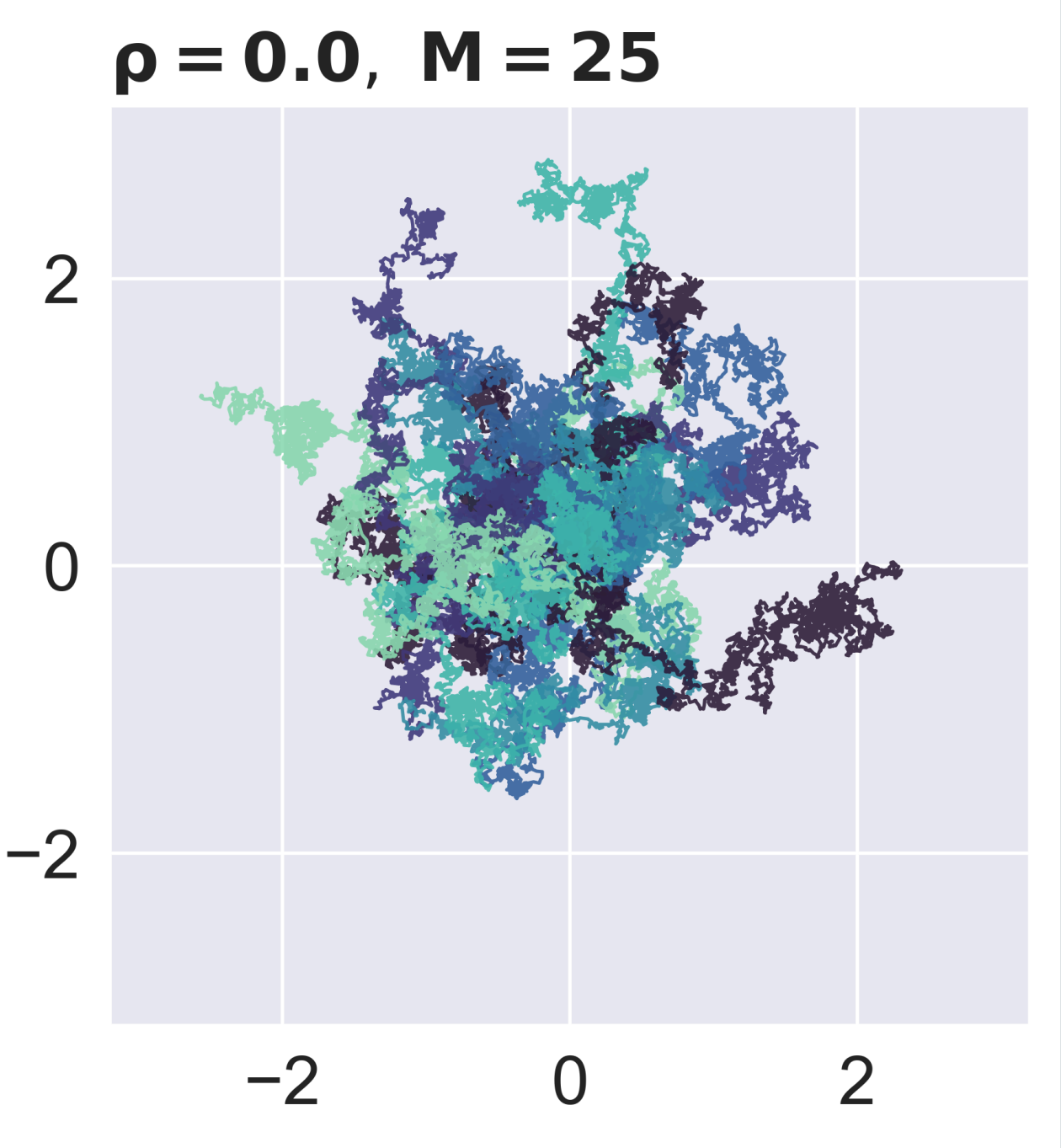

ρ = 0.0, M = 25
2
0
−2
−2
0
2

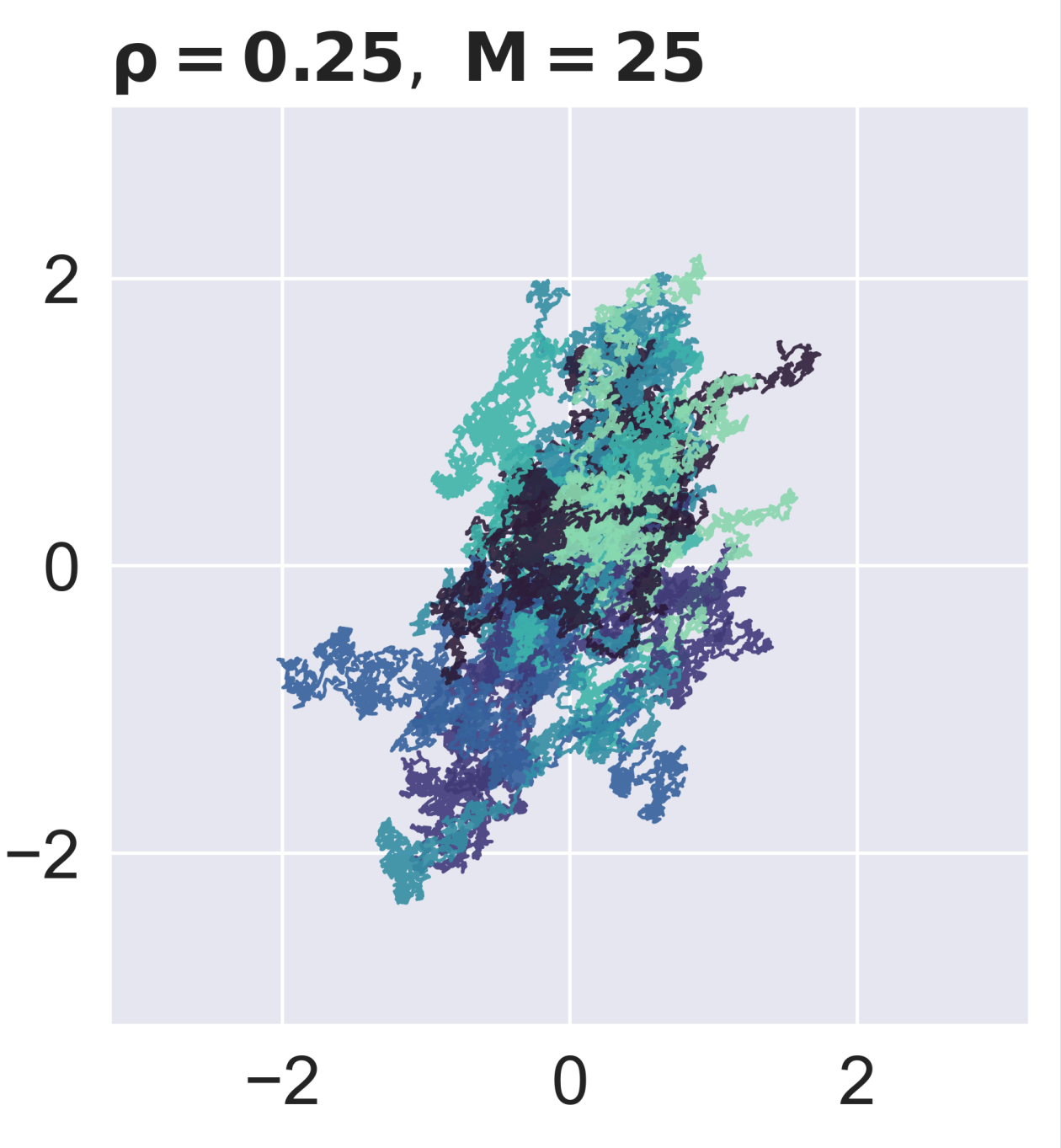

ρ = −0.99, M = 25
2
0
−2
−2
0
2

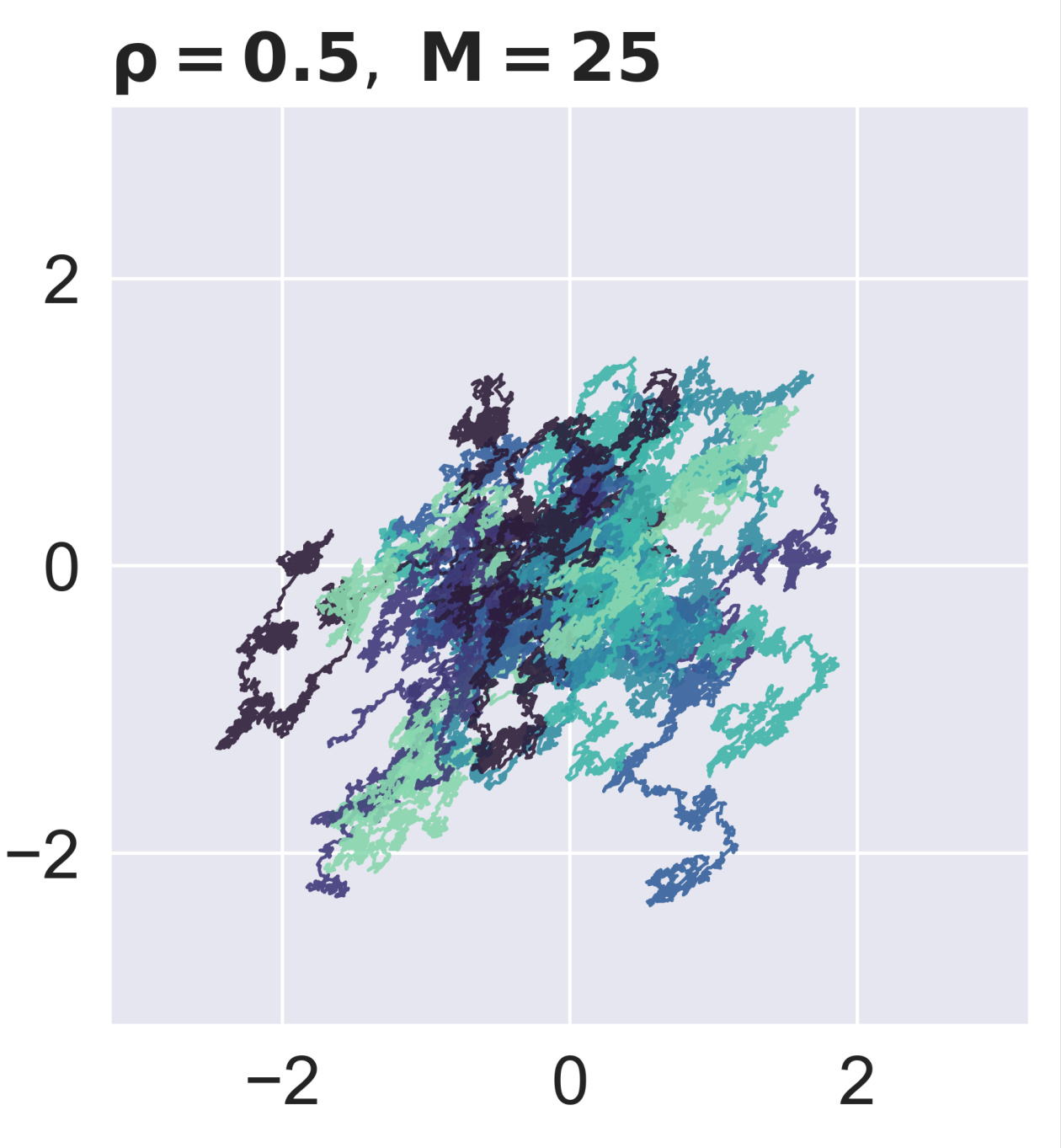

ρ = −0.75, M = 25
2
0
−2
−2
0
2

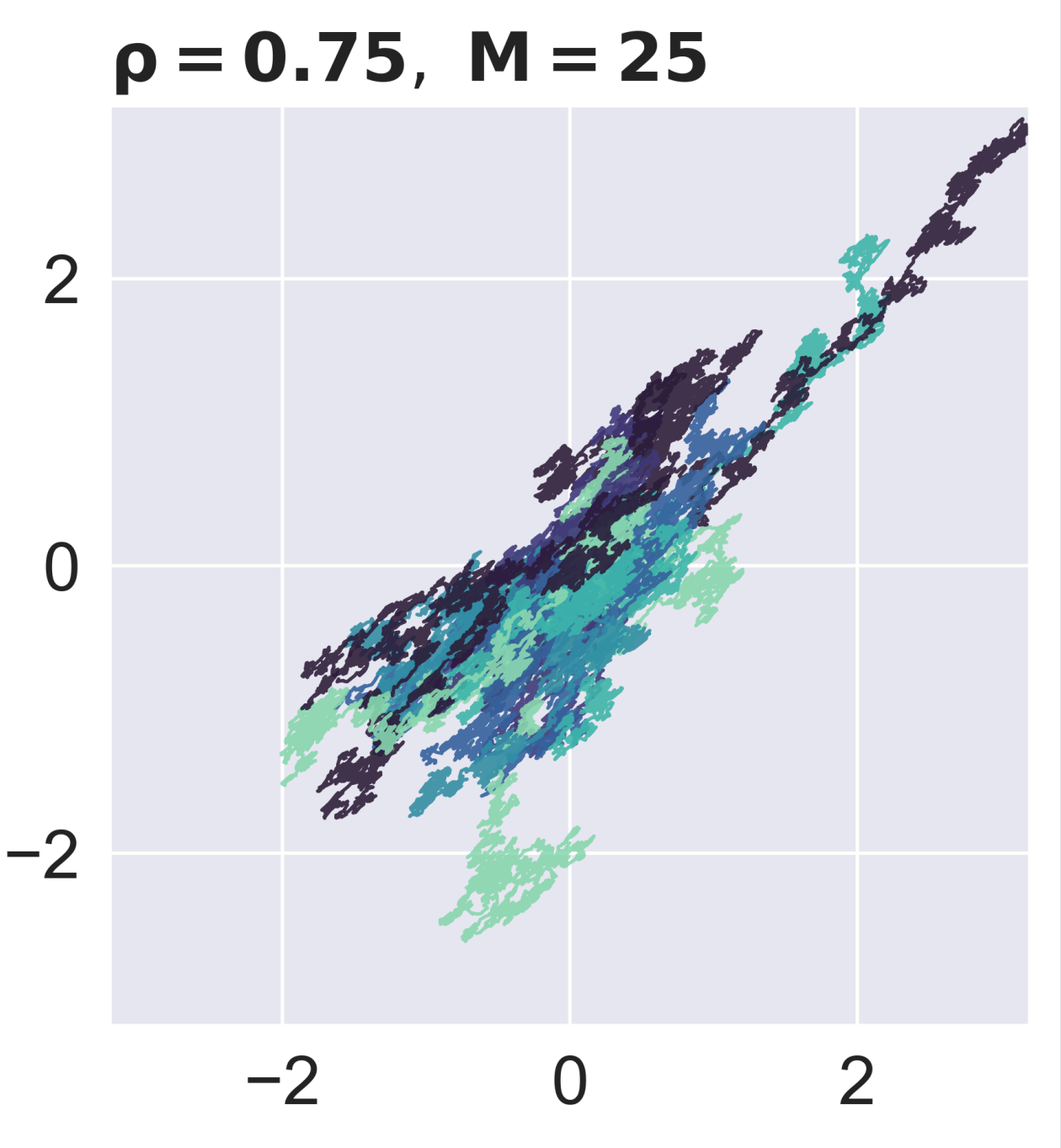

ρ = −0.5, M = 25
2
0
−2
−2
0
2

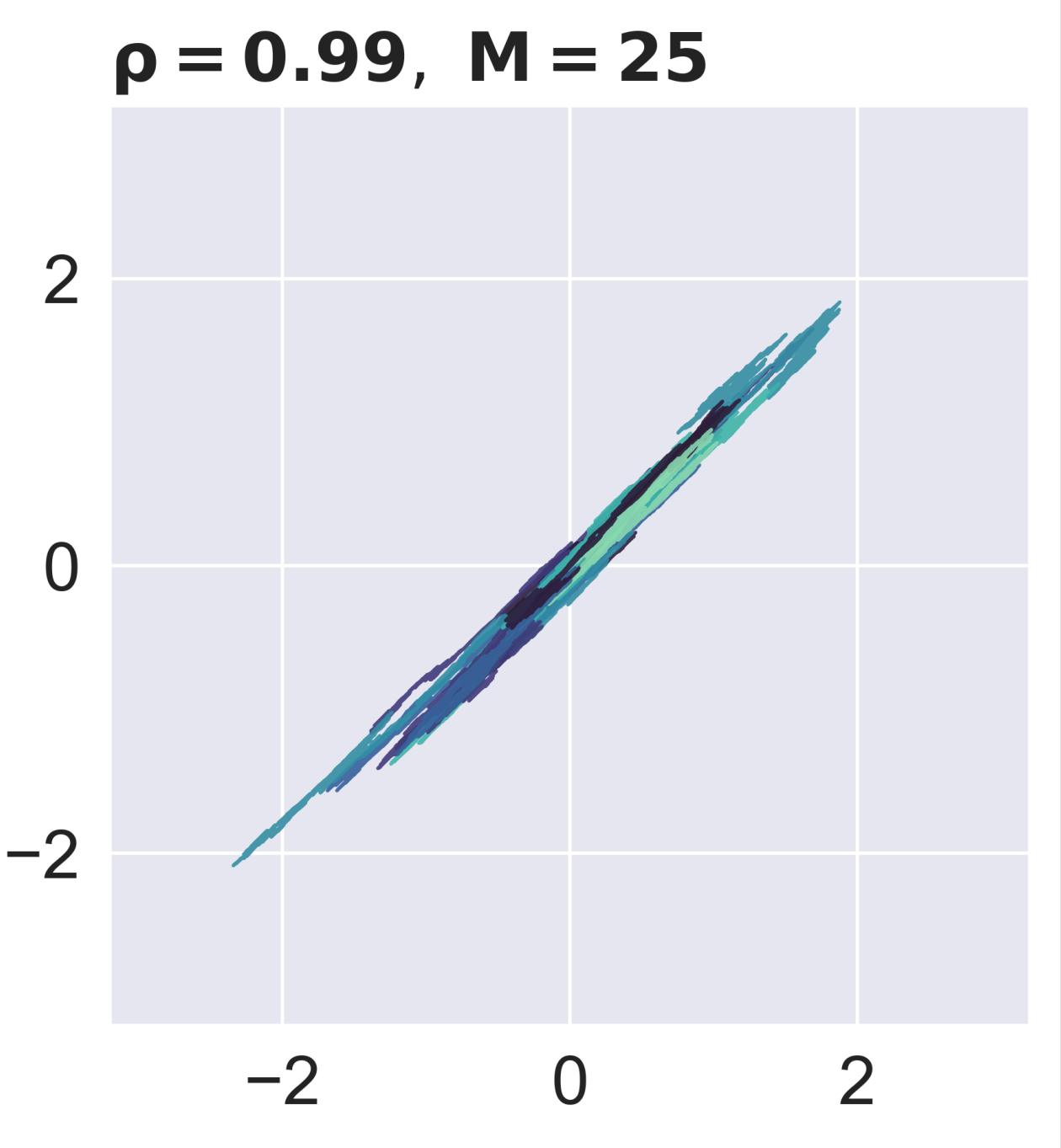

ρ = −0.25, M = 25
2
0
−2
−2
0
2


ρ = 0.0, M = 25
2
0
−2
−2
0
2


ρ = 0.25, M = 25
2
0
−2
−2
0
2


ρ = 0.5, M = 25
2
0
−2
−2
0
2


ρ = 0.75, M = 25
2
0
−2
−2
0
2


ρ = 0.99, M = 25
2
0
−2
−2
0
2

Stochastic volatility models

A one-factor **stochastic volatility model** is the solution (S, V) of a bivariate SDE of the type

$$
\begin{aligned}
dS_t &= \mu(t, S_t)dt + \sigma(t, S_t, V_t)dW_t \\
dV_t &= \alpha(t, V_t)dt + \lambda(t, V_t)dB_t \\
d\langle W, B\rangle_t &= \rho dt
\end{aligned}
$$

starting from a fixed value (S_0, V_0), where μ, σ, α and λ are real functions, and $(W, B) = (W_t, B_t)_{0 \leq t \leq T}$ is a correlated bivariate Brownian motion.

- S is the asset price process,
- V is the variance/volatility process (depending on the function σ).

Stochastic volatility model	SDEs with $d\langle W, B\rangle_t = \rho dt$
Schöbel-Zhu	$dS_t = rS_t dt + V_t S_t dW_t$ $dV_t = \kappa(\theta - V_t)\,dt + \lambda dB_t$
Heston	$dS_t = rS_t dt + \sqrt{V_t} S_t dW_t$ $dV_t = \kappa(\theta - V_t)\,dt + \lambda\sqrt{V_t} dB_t$
GARCH	$dS_t = rS_t dt + \sqrt{V_t} S_t dW_t$ $dV_t = \kappa(\theta - V_t)\,dt + \lambda V_t dB_t$
Bergomi	$dS_t = rS_t dt + e^{V_t} S_t dW_t$ $dV_t = \kappa(\theta - V_t)\,dt + \lambda dB_t$

Simulate a stochastic volatility model

The solution of a stochastic volatility model can be easily simulated using the **Euler scheme**. Let $\mathcal{T} = \{t_0, t_1, \ldots, t_N\}$ be a uniform time grid with $t_n = n\Delta_t$ and $\Delta_t = T/N$. The Euler scheme reads:

$$S_{t_{n+1}} = S_{t_n} + \mu(t_n, S_{t_n})\Delta_t + \sigma(t_n, S_{t_n}, V_{t_n})\left(\sqrt{\Delta_t} G_n\right)$$

$$V_{t_{n+1}} = V_{t_n} + \alpha(t_n, V_{t_n})\Delta_t + \lambda(t_n, V_{t_n})\sqrt{\Delta_t}\left(\rho G_n + \sqrt{1-\rho^2} H_n\right)$$

starting from a fixed value (S_0, V_0), where we use the fact that

$$W_{t_{n+1}} - W_{t_n} = \sqrt{\Delta_t} G_n$$

$$B_{t_{n+1}} - B_{t_n} = \sqrt{\Delta_t}\left(\rho G_n + \sqrt{1-\rho^2} H_n\right)$$

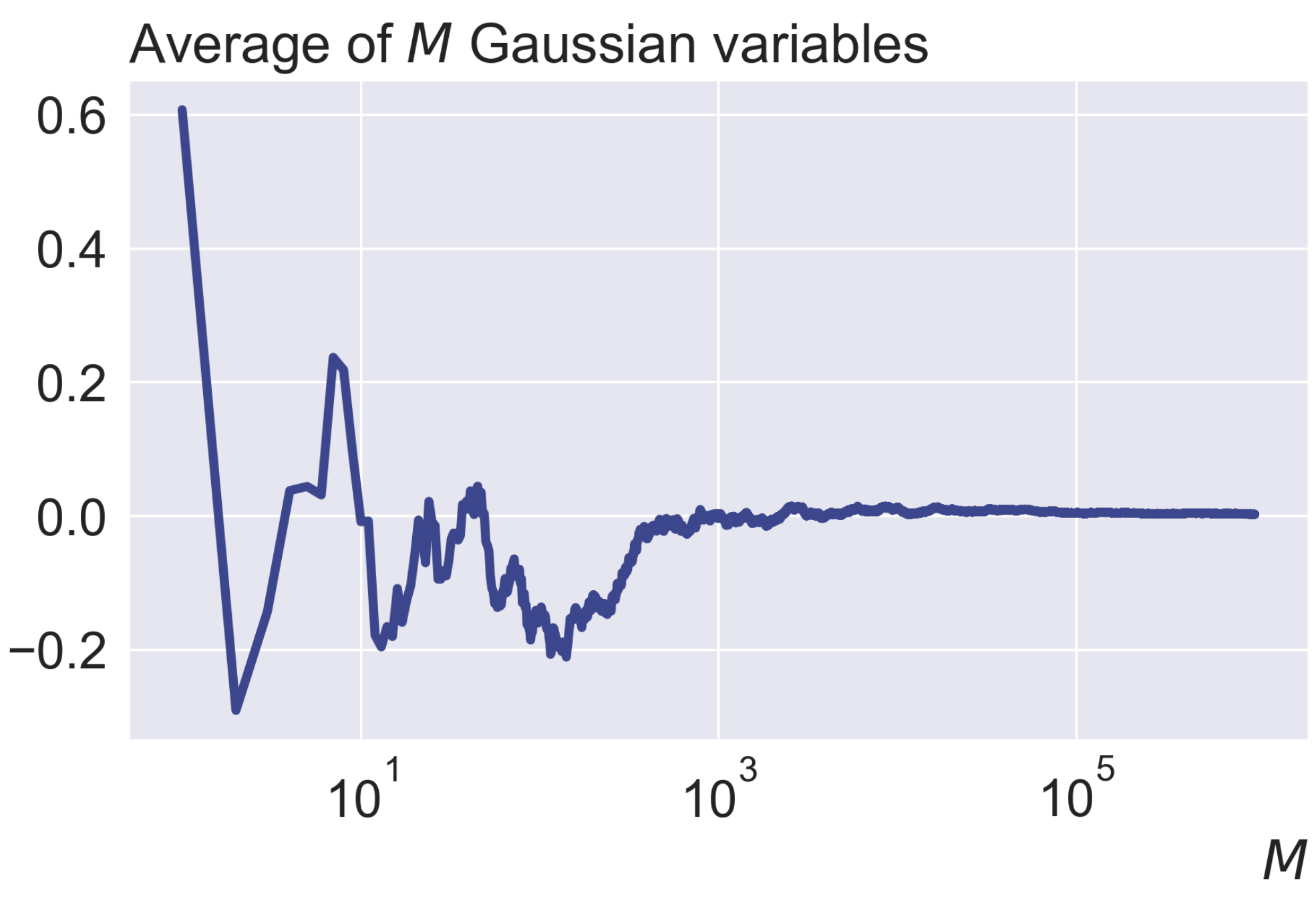
6.3 Monte Carlo method for option pricing

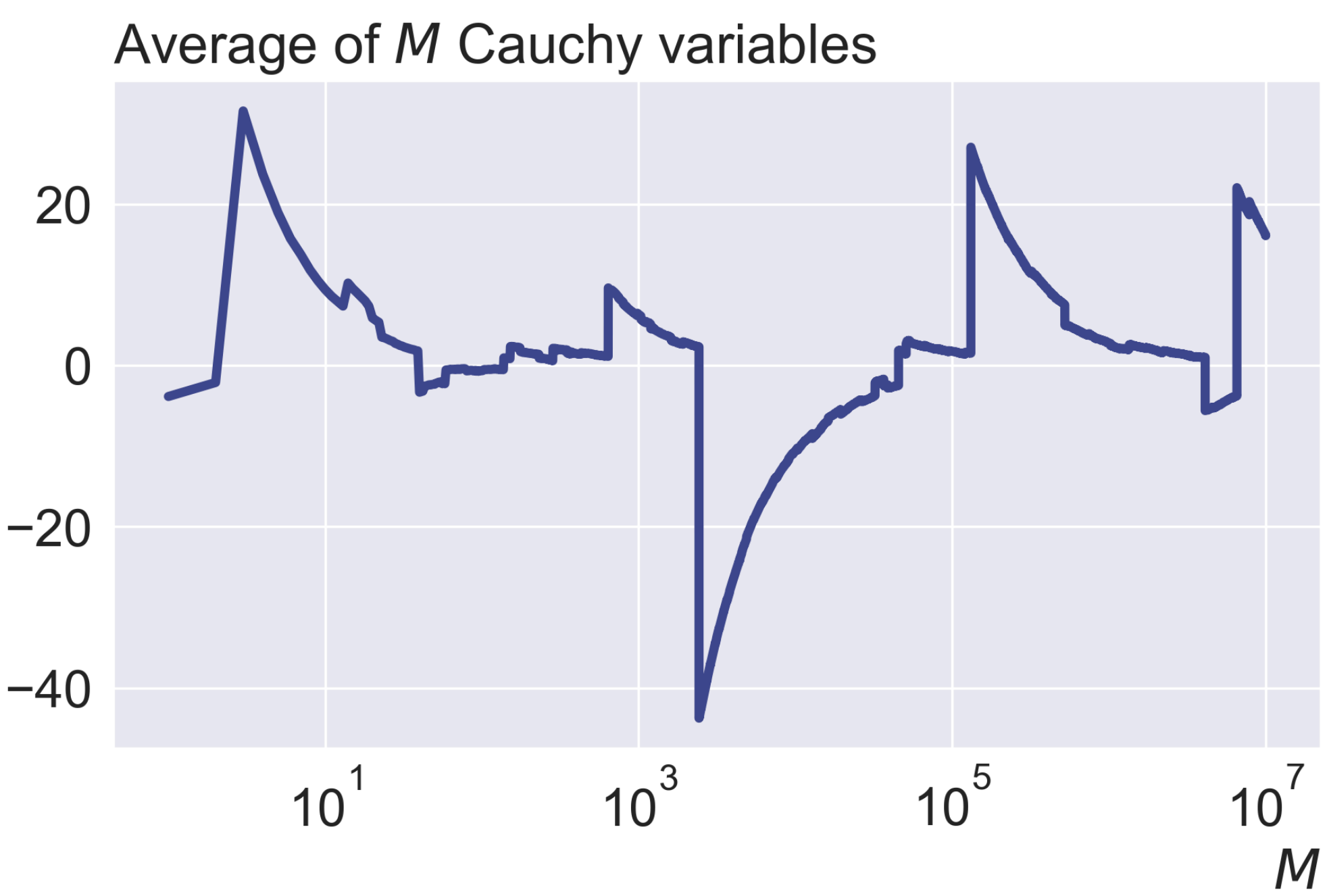
Strong law of large numbers

Let $X_1, X_2, \ldots, X_M$ be M independent and identically distributed random variables with finite expectation $\mu = \mathbb{E}[X]$. Then

$$\lim_{M \to \infty} \frac{1}{M} \sum_{m=1}^{M} X_m \xrightarrow{a.s.} \mathbb{E}[X]$$

This is the cornerstone of the Monte Carlo method, which holds as long as the mean of the random variable exists and is finite, which is the case for the vast majority of classical distributions. Often, we are interested in the expectation $\mathbb{E}[f(X)]$ of a functional f of a random variable X, which can be estimated by $\frac{1}{M} \sum_{m=1}^{M} f(X_m)$.


Average of M Gaussian variables
0.6
0.4
0.2
0.0
−0.2
10^1
10^3
10^5
M


Average of M Cauchy variables
20
0
−20
−40
10^1
10^3
10^5
10^7
M

```
import numpy as np
from scipy.stats import norm, cauchy
import matplotlib.pyplot as plt

M = 10**6 # number of Monte Carlo paths
G = norm.rvs(size=M) # cauchy.rvs(size=M)
cumsum = np.cumsum(G) # cumulative sum of simulations
cumsum2 = np.cumsum(G*G) # cumulative sum of squared simulations
Marray = 1+np.arange(M)
moment = cumsum/Marray # 1st moment
moment2 = cumsum2/Marray # 2nd moment
variance = moment2 - moment**2
stdev = np.sqrt(variance)/np.sqrt(Marray)

fig, ax = plt.subplots()
ax.semilogx(Marray,moment,linewidth=3)
plt.title("Average of $M$ Gaussian variables",loc='left')
ax.set_xlabel("$M$",loc='right')
fig.tight_layout(pad=0.0)
```

Central limit theorem

Let $X_1, X_2, \ldots, X_M$ be M independent and identically distributed random variables with finite expectation $\mu = \mathbb{E}[X]$ and finite variance $\sigma^2 = \mathbb{E}[(X - \mathbb{E}[X])^2]$. Then

$$Z := \lim_{M \to \infty} \left(\frac{\frac{1}{M} \sum_{m=1}^{M} X_m - \mu}{\sigma / \sqrt{M}} \right) \xrightarrow{\mathcal{L}} \mathcal{N}(0, 1)$$

This important result states that, with proper scaling, a Monte Carlo estimator converges to a Gaussian distribution when the number of Monte Carlo simulations is large. This result can be used to compute **asymptotic confidence intervals**.

Asymptotic confidence interval

Remark that $\sigma/\sqrt{M}$ is the standard deviation of $\frac{1}{M}\sum_{m=1}^{M} X_m$, since

$$\mathbb{V}\mathrm{ar}\left[\frac{1}{M}\sum_{m=1}^{M} X_m\right] = \frac{1}{M^2}\mathbb{V}\mathrm{ar}\left[\sum_{m=1}^{M} X_m\right] = \frac{1}{M^2} M\,\mathbb{V}\mathrm{ar}\left[X\right] = \frac{\sigma^2}{M}$$

Let Φ be the CDF of the standard Gaussian variable. Let $0 < \varepsilon \ll 1$. A centered confidence interval containing the central $1-\varepsilon\%$ of a standard Gaussian distribution is given by

$$[\Phi^{-1}(\varepsilon/2), \Phi^{-1}(1-\varepsilon/2)]$$

Asymptotic confidence interval

Indeed, if $Z \sim \mathcal{N}(0,1)$ and Φ is the standard Gaussian CDF, then

$$\begin{aligned} & \mathbb{P}(Z \in [\Phi^{-1}(\varepsilon/2), \Phi^{-1}(1-\varepsilon/2)]) \\ = \; & \mathbb{P}(\Phi^{-1}(\varepsilon/2) \leq Z \leq \Phi^{-1}(1-\varepsilon/2)) \\ = \; & \mathbb{P}(\varepsilon/2 \leq \Phi(Z) \leq 1-\varepsilon/2) \\ = \; & \mathbb{P}(\Phi(Z) \leq 1-\varepsilon/2) - \mathbb{P}(\Phi(Z) \leq \varepsilon/2) \end{aligned}$$

Using the fact that $\Phi(Z) \sim \mathcal{U}(0,1)$ (proof next slide), the above is the difference between two evaluations of the uniform CDF F_U

$$\begin{aligned} = \; & F_U(1-\varepsilon/2) - F_U(\varepsilon/2) \\ = \; & 1-\varepsilon/2-\varepsilon/2 \; = \; 1-\varepsilon \end{aligned}$$

Proof that $F_X(X) \sim \mathcal{U}(0,1)$

Let X be a random variable and let $F_X(x) = \mathbb{P}(X \leq x)$ be the CDF of X. The the CDF of the random variable $F_X(X)$ is given by

$$\begin{aligned} \mathbb{P}(F_X(X) \leq u) &= \mathbb{P}(X \leq F_X^{-1}(u)) \\ &= F_X(F_X^{-1}(u)) = u \end{aligned}$$

which is exactly the definition of the CDF of a standard uniform random variable. This proves that the distribution of $F_X(X)$ is uniform: $F_X(X) \sim \mathcal{U}(0,1)$. In the case when X is a standard Gaussian variable, we rather use the notation $Z \sim \mathcal{N}(0,1)$ for the random variable, and Φ for the CDF of Z. In other words, $\Phi(Z) \sim \mathcal{U}(0,1)$, as used in the previous slide.

Asymptotic confidence interval

Using this interval $[\Phi^{-1}(\varepsilon/2), \Phi^{-1}(1-\varepsilon/2)]$, we get

$$\mathbb{P}\left(\Phi^{-1}(\varepsilon/2) \leq Z \leq \Phi^{-1}(1-\varepsilon/2)\right) = 1-\varepsilon$$

$$\mathbb{P}\left(\Phi^{-1}(\varepsilon/2) \leq \frac{\frac{1}{M}\sum_{m=1}^{M} X_m - \mu}{\sigma/\sqrt{M}} \leq \Phi^{-1}(1-\varepsilon/2)\right) = 1-\varepsilon$$

and finally

$$\mathbb{P}\left(\mu + \frac{\sigma}{\sqrt{M}}\Phi^{-1}(\varepsilon/2) \leq \frac{1}{M}\sum_{m=1}^{M} X_m \leq \mu + \frac{\sigma}{\sqrt{M}}\Phi^{-1}(1-\varepsilon/2)\right)$$

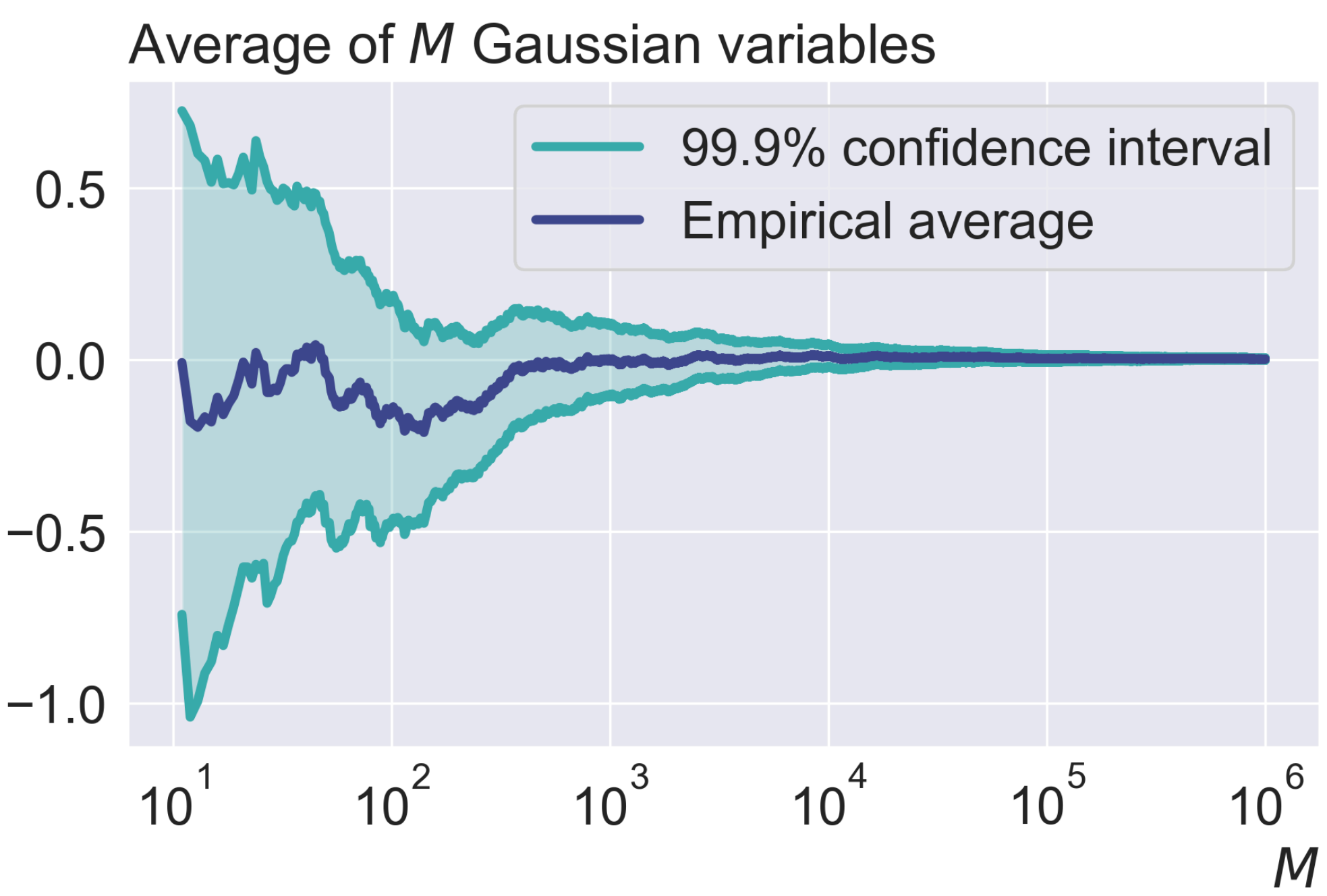
Asymptotic confidence interval

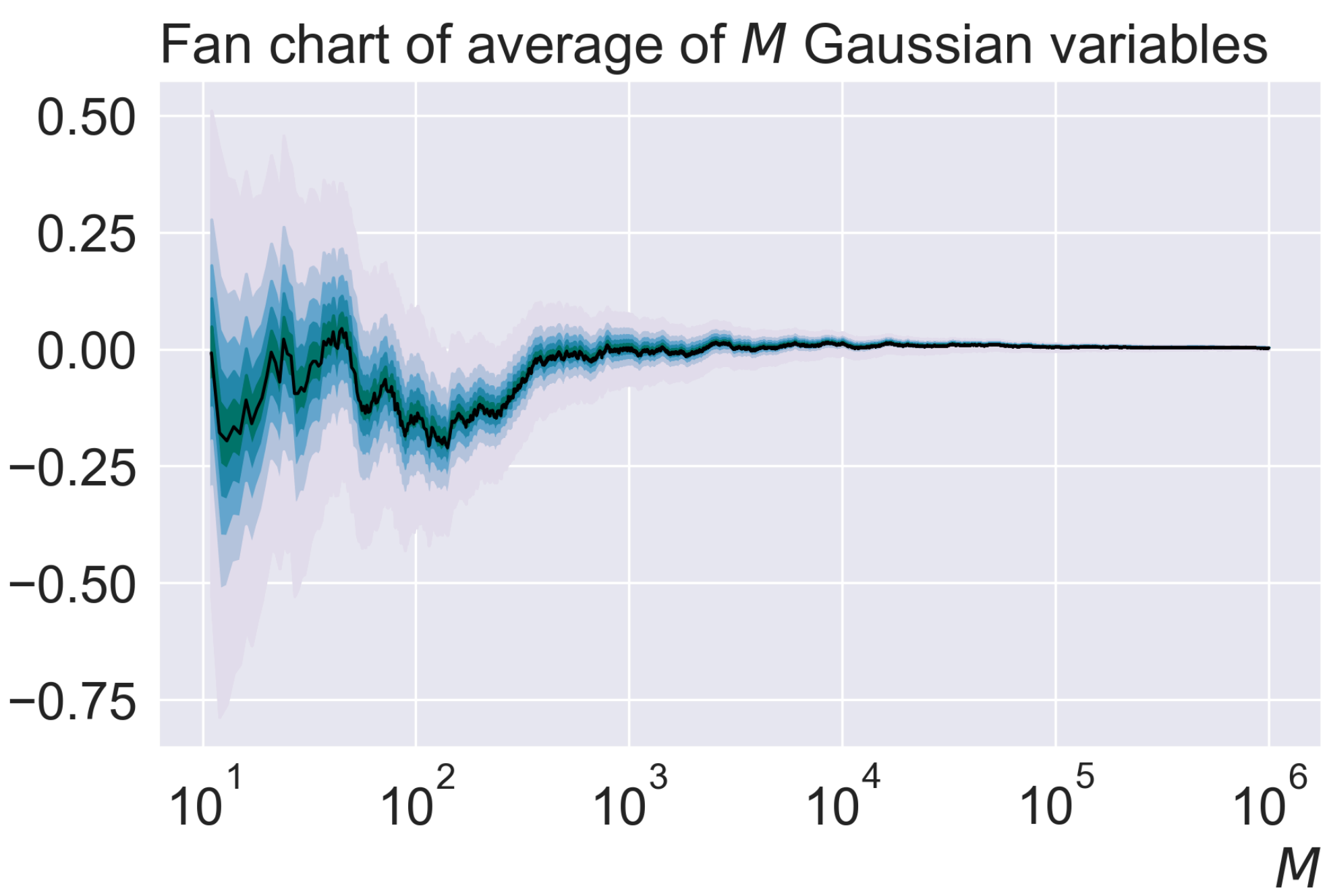
The limitations of this confidence interval are that:

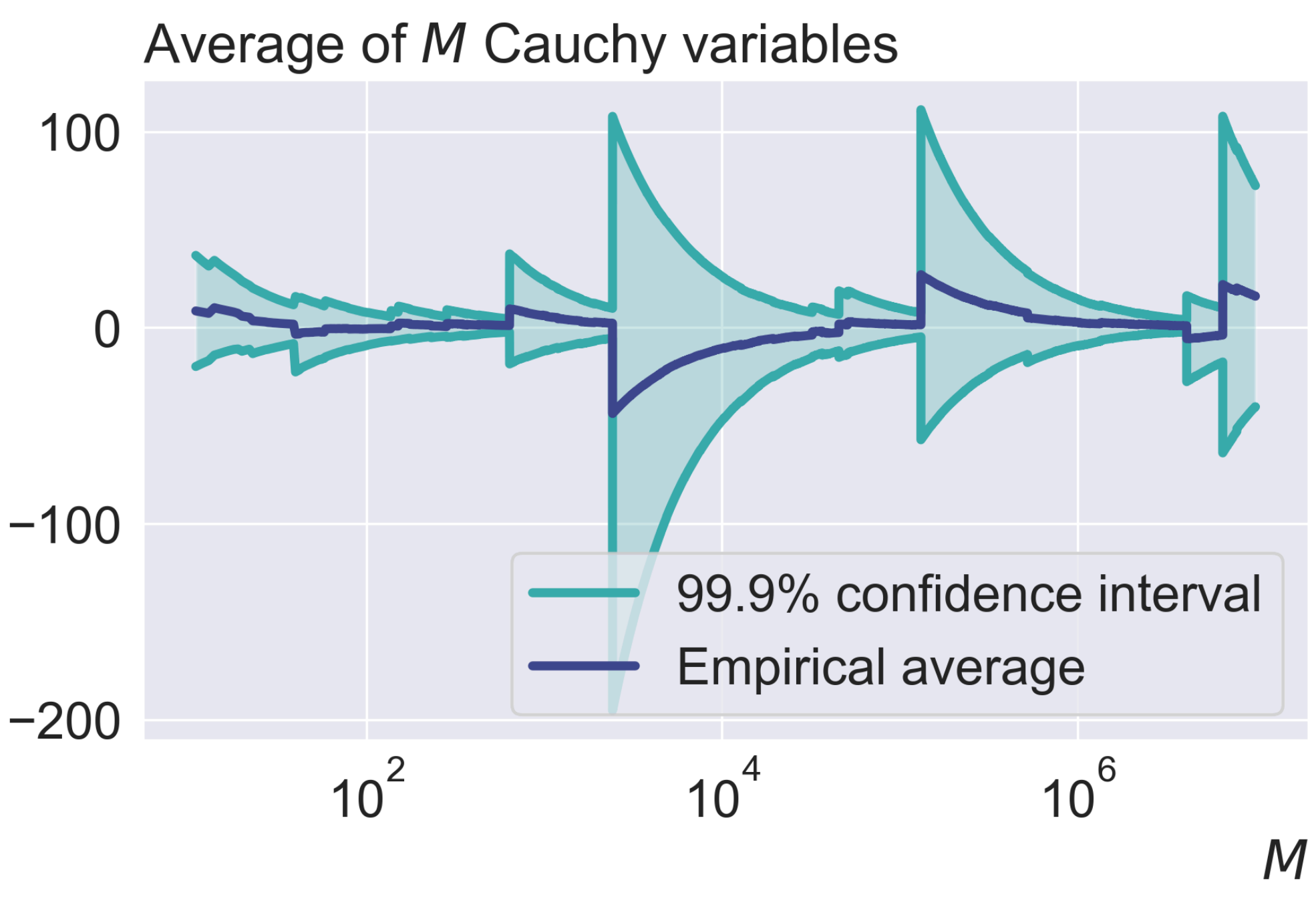
▷ It is only *asymptotic*. The Gaussian distribution limit is only achieved for $M \to \infty$; for fixed M it is only an approximation. The larger M and the closer the distribution of X to a Gaussian, the better this approximation.

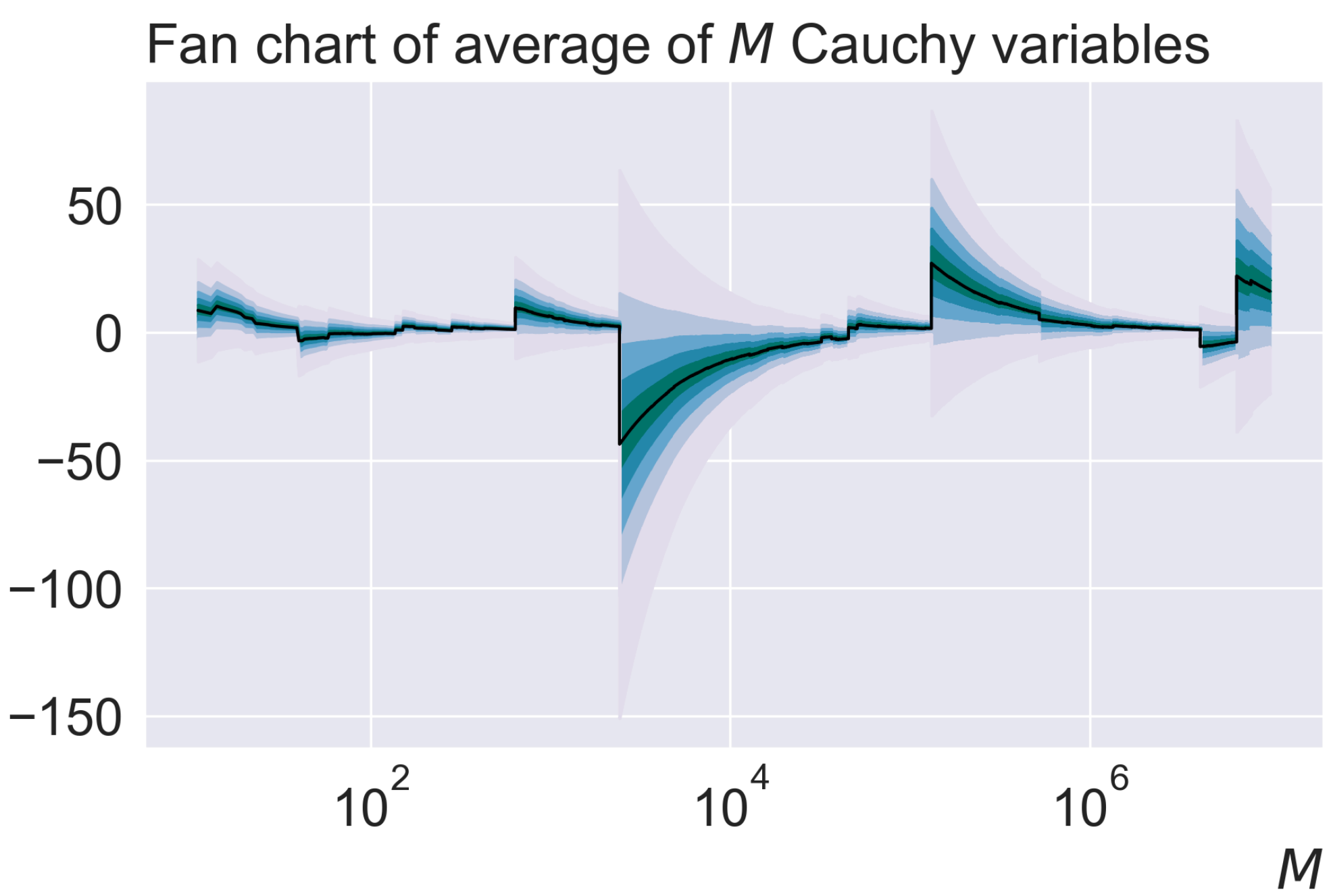
▷ It requires the knowledge of the *exact* mean μ and the *exact* standard deviation σ. Often these are unknown and have to be estimated from the dataset. This confidence interval is only approximate if μ and σ are replaced by empirical estimates.

Nevertheless, the main information provided by the central limit theorem is that Monte Carlo estimators converge at a speed proportional to $\sigma/\sqrt{M}$.


Average of M Gaussian variables
99.9% confidence interval
Empirical average
0.5
0.0
−0.5
−1.0
10^1
10^2
10^3
10^4
10^5
10^6
M


Fan chart of average of M Gaussian variables
0.50
0.25
0.00
−0.25
−0.50
−0.75
10^1
10^2
10^3
10^4
10^5
10^6
M


Average of M Cauchy variables
100
0
−100
−200
10^2
10^4
10^6
M
99.9% confidence interval
Empirical average


Fan chart of average of M Cauchy variables
50
0
−50
−100
−150
10^2
10^4
10^6
M

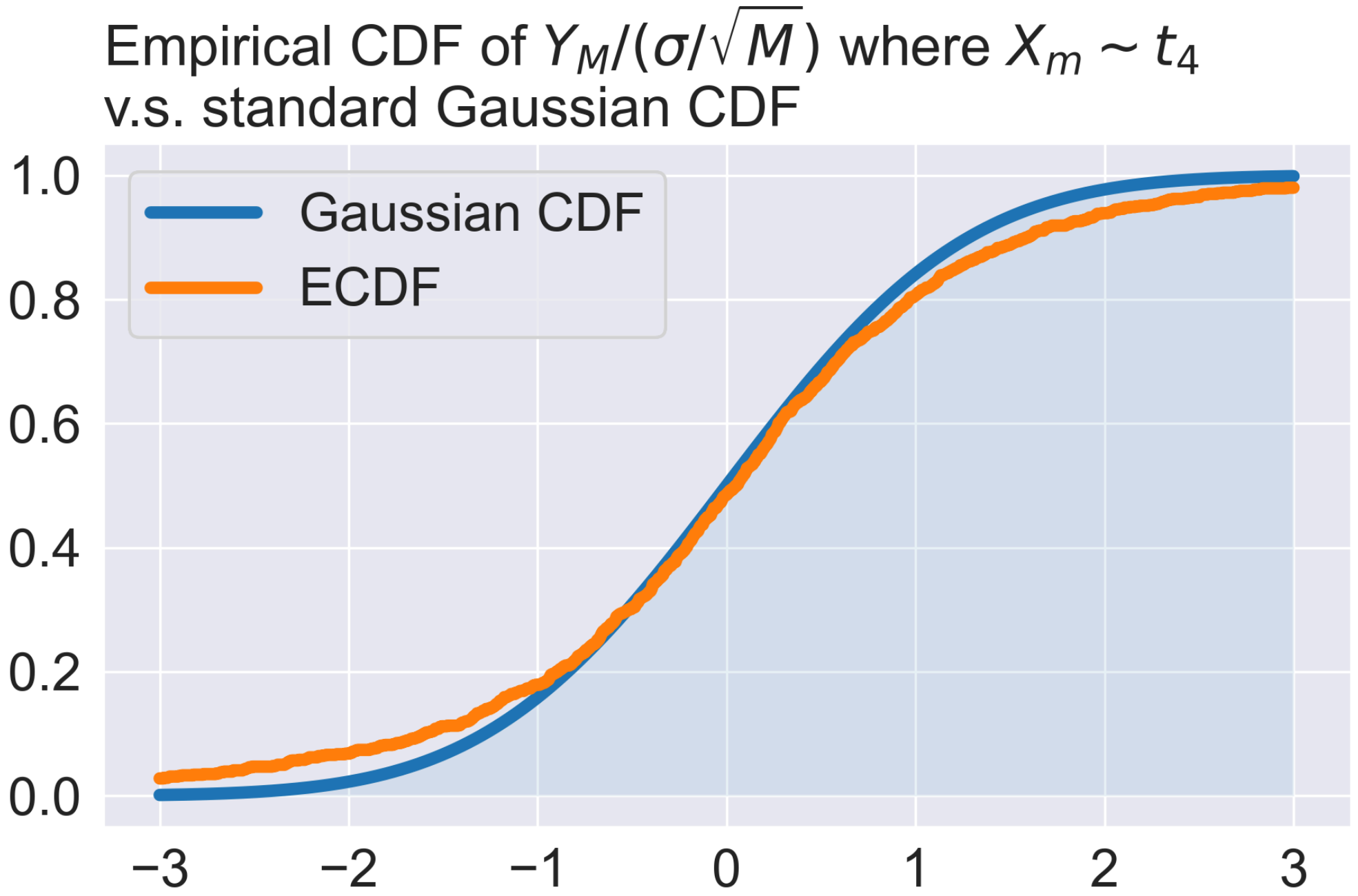
Berry-Esseen theorem

The empirical CDF of a sample $Z_1, Z_2, \ldots, Z_M$ is defined by $F_M(z) := \frac{1}{M} \sum_{m=1}^{M} 1\{Z_m \leq z\}$.

Let $X_1, X_2, \ldots, X_M$ be M independent and identically distributed random variables with $\mathbb{E}[X] = 0$, $\mathbb{E}[X^2] = \sigma^2 > 0$ and $\mathbb{E}[|X|^3] = \rho < \infty$. Define $Y_M := \frac{1}{M} \sum_{m=1}^{M} X_m$ the sample mean, and let F_M be the empirical CDF of $Y_M / (\sigma / \sqrt{M})$. Then there exists $C > 0$ such that

$$|F_M(x) - \Phi(x)| \leq \frac{C\rho}{\sigma^3 \sqrt{M}}$$

for all $x \in \mathbb{R}$, where Φ is the standard Gaussian CDF. This result can be used to assess the convergence speed of the central limit theorem.


Empirical CDF of $Y_M/(\sigma/\sqrt{M})$ where $X_m \sim t_4$
v.s. standard Gaussian CDF
Gaussian CDF
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
−3
−2
−1
0
1
2
3

Monte Carlo foundations: summary

	Theorem	Variables	Condition
Law of large numbers	$\lim_{M\to\infty} \frac{1}{M}\sum_{m=1}^{M} X_m \xrightarrow{a.s.} \mathbb{E}[X]$	$X_1, X_2, \ldots, X_M$ i.i.d. sample	$\mathbb{E}[\lvert X\rvert] < \infty$
Central limit theorem	$\lim_{M\to\infty} \left(\frac{\frac{1}{M}\sum_{m=1}^{M} X_m - \mu}{\sigma/\sqrt{M}} \right) \xrightarrow{\mathcal{L}} \mathcal{N}(0,1)$	$\mu = \mathbb{E}[X]$ $\sigma^2 = \mathbb{V}\mathrm{ar}[X]$	$\mathbb{E}[\lvert X\rvert^2] < \infty$
Berry-Esseen theorem	$\lvert F_M(x) - \Phi(x)\rvert \leq \frac{C\rho}{\sigma^3\sqrt{M}}$	$\rho = \mathbb{E}[\lvert X\rvert^3]$ $Y_M = \frac{1}{M}\sum_{m=1}^{M} X_m$ F_M ECDF of $Y_M/(\sigma/\sqrt{M})$, Φ CDF of $\mathcal{N}(0,1)$ $C \in [0.409, 0.475]$	$\mathbb{E}[X] = 0$ $\mathbb{E}[\lvert X\rvert^3] < \infty$

Monte Carlo option pricing

Let $(S_{t_n}^m)_{n=0,\dots,N}^{m=1,\dots,M}$ be M Monte Carlo simulations of a stochastic model $S = (S_t)_{0 \le t \le T}$. Let $f(S_T)$ be the final payoff of a European option, for example $f(S_T) = (S_T - K)^+$ (call option) or $f(S_T) = (K - S_T)^+$ (put option). A Monte Carlo estimator of the price $P_0 = \mathbb{E}[e^{-rT} f(S_T)]$ is given by

$$P_0^M := \frac{1}{M} \sum_{m=1}^{M} e^{-rT} f(S_T^m)$$

If the stochastic model S is such that P_0 exists and is finite, then the strong law of large numbers applies and ensures that $\lim_{M \to \infty} P_0^M \xrightarrow{a.s.} P_0$.

Monte Carlo pricing: convergence speed

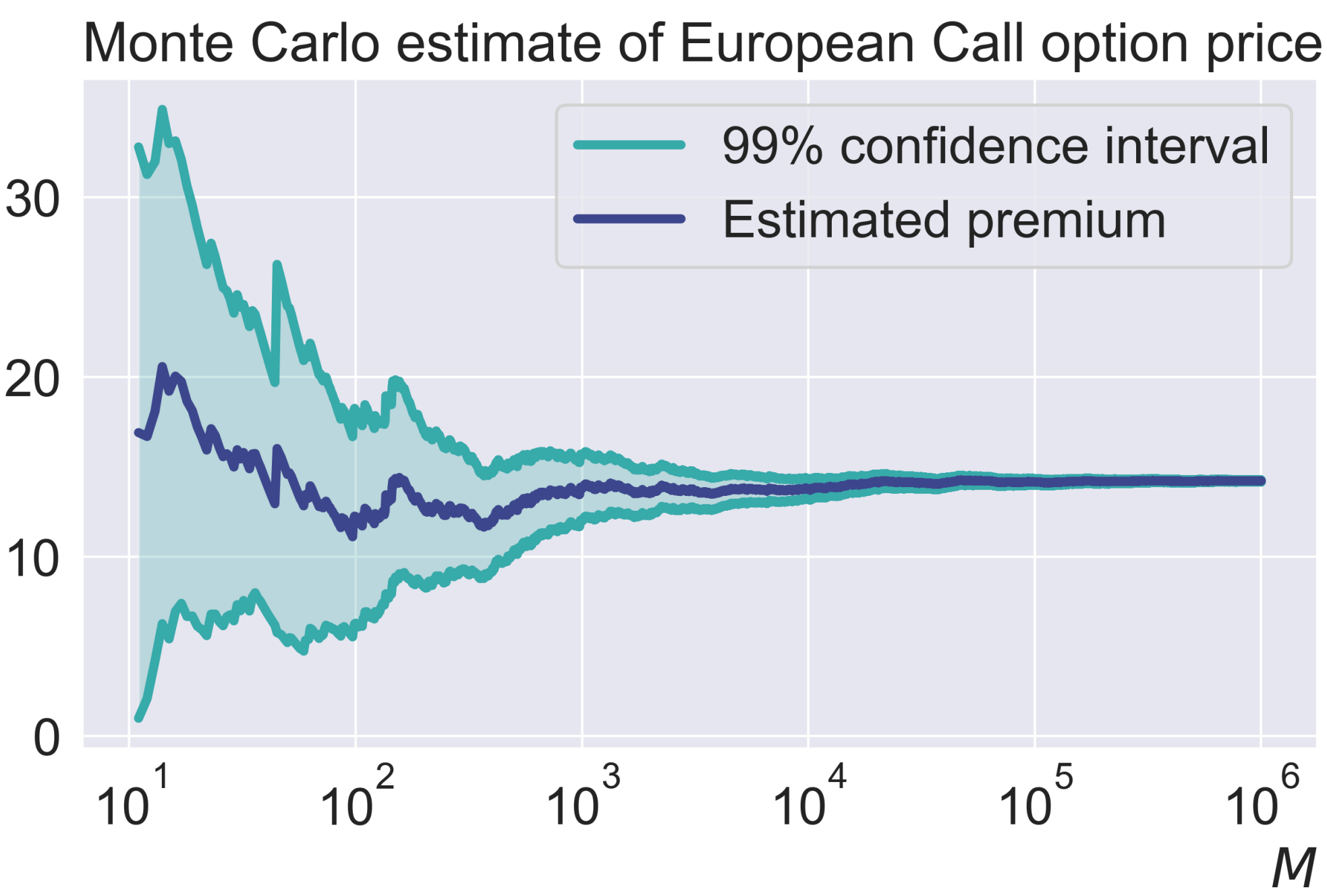
Let $\sigma^2 = \mathbb{V}\mathrm{ar}[e^{-rT} f(S_T)]$. We know that an asymptotic $1-\varepsilon\%$ confidence interval for P_0^M is

$$\left[P_0 + \frac{\sigma}{\sqrt{M}} \Phi^{-1}(\varepsilon/2) \ , \ P_0 + \frac{\sigma}{\sqrt{M}} \Phi^{-1}(1-\varepsilon/2) \right]$$

The smaller this interval, the better. It can be shortened by:

▷ **Increasing** $\mathbf{M}$. It is simple and safe but the computational cost of the Monte Carlo algorithm is $\mathcal{O}(M)$, and halving the interval requires to multiply M by 4.

▷ **Reducing** σ. There exist many techniques known as **variance reduction techniques** to reduce the variance of a Monte Carlo estimator. They are sometimes specific to one type of payoff.


Monte Carlo estimate of European Call option price
99% confidence interval
Estimated premium
30
20
10
0
10^1
10^2
10^3
10^4
10^5
10^6
M

Examples of exotic financial options

Asian option: call example: $\text{payoff} = \max(S_T - \frac{1}{T}\int_0^T S_t dt, 0)$

Lookback option: call example: $\text{payoff} = S_T - \min_{0\le t\le T} S_t$

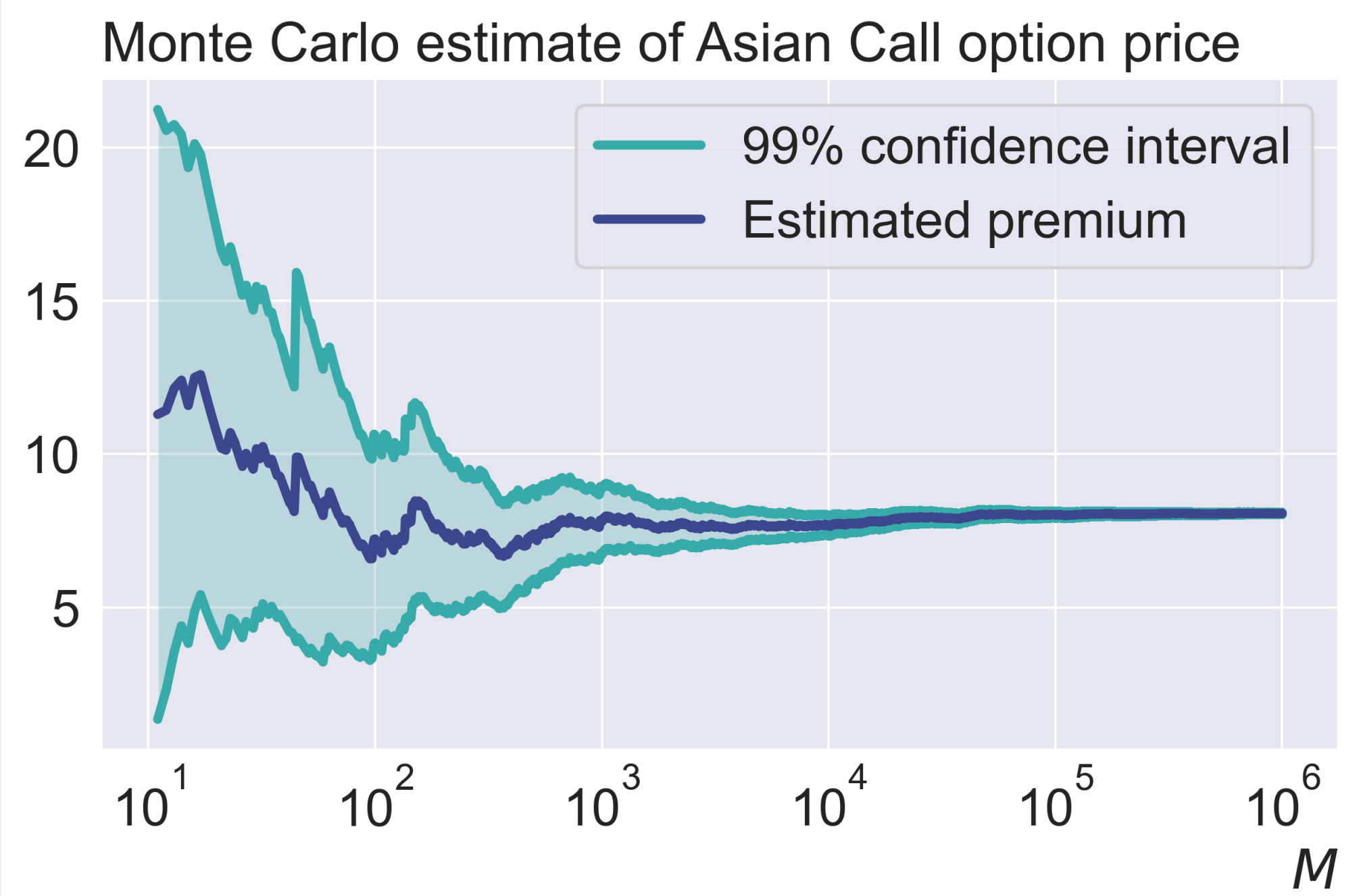
Rainbow option: call on max: $\text{payoff} = \max(\max(S_{1,T}, S_{2,T}) - K, 0)$

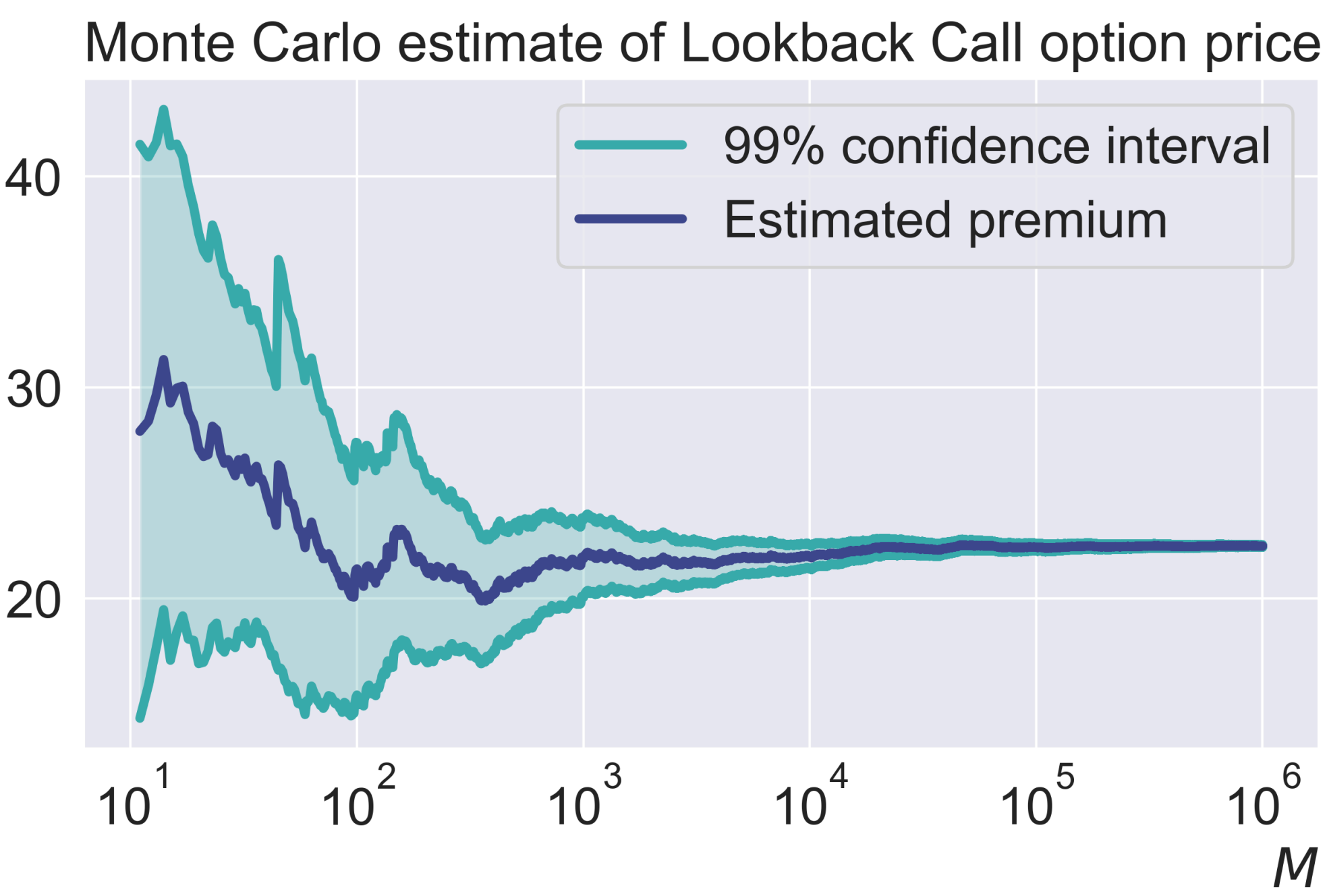
Basket option: call example: $\text{payoff} = \max((S_{1,T} + S_{2,T})/2 - K, 0)$

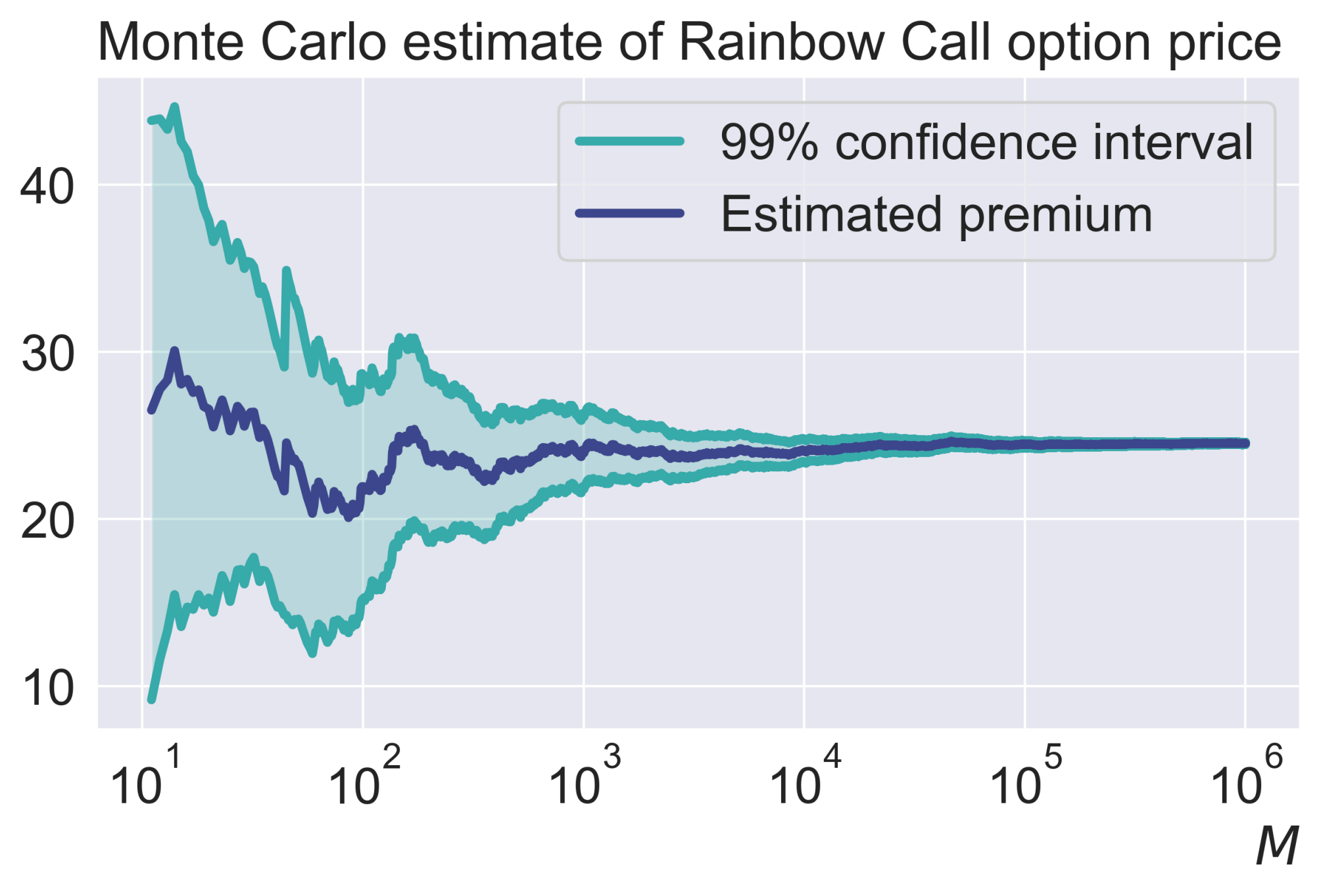
Barrier option: up-and-out call: $\max(S_T - K, 0)\mathbf{1}(\max_{0\le t\le T} S_t \le B)$

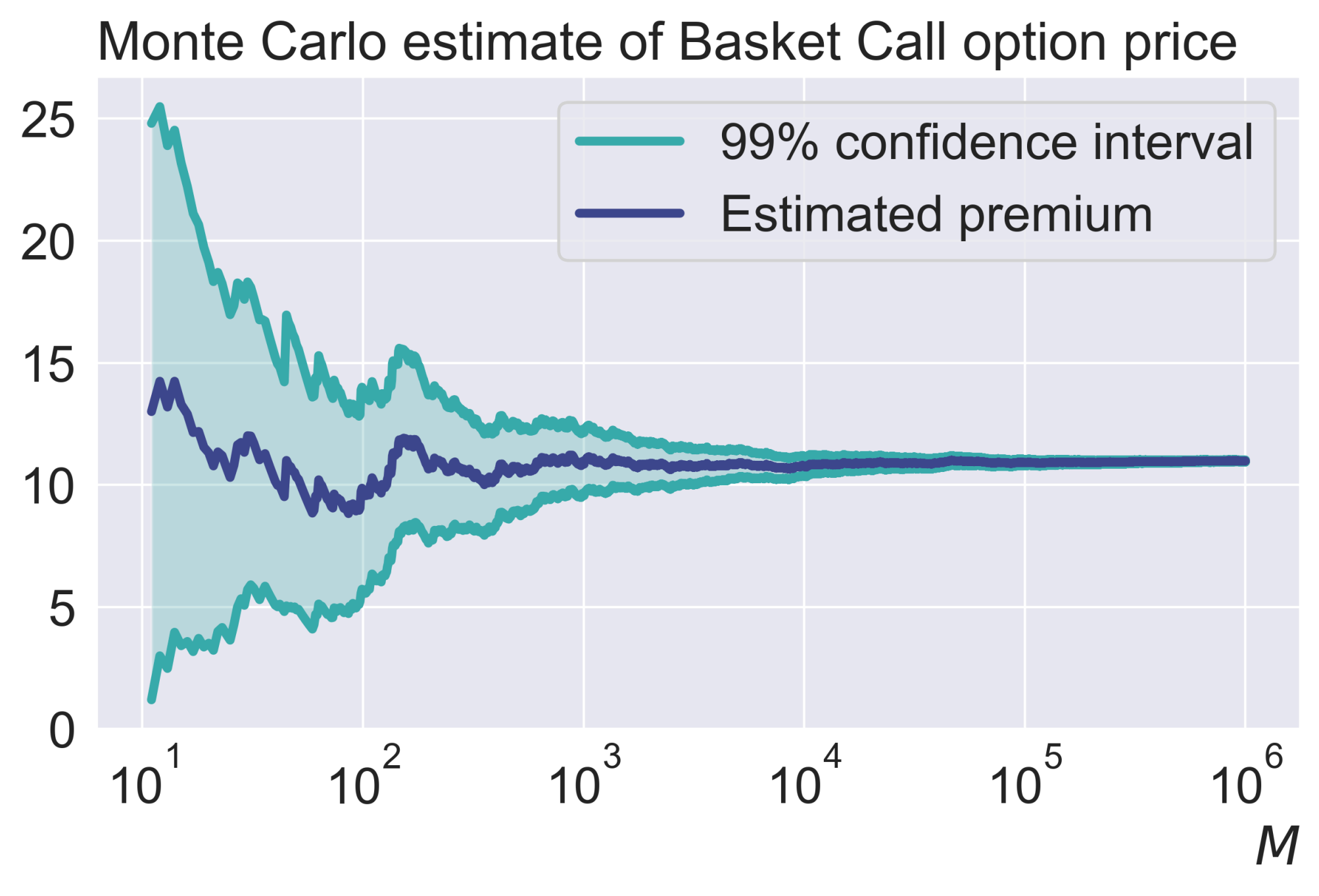
TARF option = Target Accrual Redemption Forward

$$P = \mathbb{E}\left[\sum_{n=1}^{N} e^{-rt_n}\left(S_{t_n} - K\right)\mathbf{1}\left\{\sum_{\ell=1}^{n}\max\left(S_{t_\ell} - K, 0\right) < \text{Target}\right\}\right]$$


Monte Carlo estimate of Asian Call option price
99% confidence interval
Estimated premium
20
15
10
5
10^1
10^2
10^3
10^4
10^5
10^6
M


Monte Carlo estimate of Lookback Call option price
99% confidence interval
Estimated premium
40
30
20
10^1
10^2
10^3
10^4
10^5
10^6
M


Monte Carlo estimate of Rainbow Call option price
99% confidence interval
Estimated premium
40
30
20
10
10^1
10^2
10^3
10^4
10^5
10^6
M


Monte Carlo estimate of Basket Call option price
99% confidence interval
Estimated premium
25
20
15
10
5
0
10^1
10^2
10^3
10^4
10^5
10^6
M